



3D TILES SPECIFICATION

COMMUNITY STANDARD

CANDIDATE PUBLIC RFC DRAFT

Version: 2.0

Submission Date: 2026-09-02

Approval Date: XXX

Publication Date:

Editor: Patrick Cozzi, Sean Lilley, Amanda Morgan, Adam Morris

Notice for Drafts: This document is not an OGC Standard. This document is distributed for review and comment. This document is subject to change without notice and may not be referred to as an OGC Standard.

Recipients of this document are invited to submit, with their comments, notification of any relevant patent rights of which they are aware and to provide supporting documentation.

License Agreement

Use of this document is subject to the license agreement at <https://www.ogc.org/license>

Copyright notice

Copyright © 2026 Open Geospatial Consortium

To obtain additional rights of use, visit <https://www.ogc.org/legal>

Note

Attention is drawn to the possibility that some of the elements of this document may be the subject of patent rights. The Open Geospatial Consortium shall not be held responsible for identifying any or all such patent rights.

Recipients of this document are requested to submit, with their comments, notification of any relevant patent claims or other intellectual property rights of which they may be aware that might be infringed by any implementation of the standard set forth in this document, and to provide supporting documentation.

CONTENTS

I. ABSTRACT	v
II. KEYWORDS	v
III. PREFACE	vi
IV. SECURITY CONSIDERATIONS	vii
V. SUBMITTING ORGANIZATIONS	viii
VI. SUBMITTERS	viii
VII. FUTURE WORK	viii
1. SCOPE	2
2. CONFORMANCE	4
2.1. Conformance Class: Core	4
2.2. Conformance Class: Coordinate Reference System	4
2.3. Conformance Class: Implicit Tiling	5
2.4. Conformance Class: 3D Gaussian Splatting Tiles Content Type	5
2.5. Conformance Class: Architecture, Engineering, and Construction Content Type	6
2.6. Conformance Class: Time-dynamic Tiles Content Type	7
2.7. Conformance Class: Vector Tiles Content Type	7
2.8. Conformance Class: Voxel Tiles Content Type	8
3. NORMATIVE REFERENCES	11
4. TERMS AND DEFINITIONS	15
5. CONVENTIONS	22
6. RELATIONSHIP TO GLTF	24
7. 3D TILES FORMAT SPECIFICATION	26
7.1. Introduction	26
7.2. Alignment with glTF	28
7.3. Concepts	29
7.4. Declarative styling specification	29
7.5. Compatibility with previous versions of 3D Tiles	29

8.	GLTF EXTENSIONS FOR 3D TILES	31
8.1.	Tileset Extensions	31
8.2.	Content Extensions	32
9.	3D METADATA SPECIFICATION	36
9.1.	Overview	36
9.2.	Concepts	36
9.3.	Schemas	37
9.4.	Storage Formats	47
10.	STYLING	57
10.1.	Overview	57
10.2.	Concepts	58
10.3.	Expressions	61
10.4.	File extension and media type	81
	ANNEX A ANNEX A (NORMATIVE) 3DTILES_TILESET	83
A.1.	Contributors	83
A.2.	Status	83
A.3.	Dependencies	83
A.4.	Required	83
A.5.	Contents	83
A.6.	Overview	85
A.7.	File Extensions	85
A.8.	Constraints	85
A.9.	Concepts	85
A.10.	Supporting Extensions	104
A.11.	Appendix A: Spatial Data Structures	108
A.12.	Schema	114
	ANNEX B CONTRIBUTORS	116
	ANNEX C REVISION HISTORY	119

I

ABSTRACT



This document describes the specification for 3D Tiles, an open standard for streaming massive heterogeneous 3D geospatial datasets.

II

KEYWORDS

The following are keywords to be used by search engines and document catalogues.

terrain, geospatial, gis, point cloud, spatial data, vector data, photogrammetry, gltf, 3d models, 3d tiles, metadata, implicit tiling, voxels, gaussian splatting, aec, time dynamic



PREFACE

Bringing techniques from graphics research, the movie industry, and the game industry to 3D geospatial, 3D Tiles defines a spatial data structure built on glTF designed for 3D, and optimized for streaming and rendering.



SECURITY CONSIDERATIONS

No security considerations have been made for this document.

V

SUBMITTING ORGANIZATIONS

The following organizations submitted this Document to the Open Geospatial Consortium (OGC):

- Bentley Systems Inc.

VI

SUBMITTERS

NAME	AFFILIATION	OGC MEMBER
Patrick Cozzi	Bentley Systems, Inc.	Yes
Sean Lilley	Bentley Systems, Inc.	Yes
Amanda Morgan	Bentley Systems, Inc.	Yes
Adam Morris	Bentley Systems, Inc.	Yes

VII

FUTURE WORK

The 3D Tiles community anticipates that revisions to this Community Standard will be required to prescribe content appropriate to meet new use cases. These use cases may arise from either (or both) the external user and developer community or from OGC review and comments. Further, future revisions will be driven by any submitted change requests that document community uses cases and requirements.



1

SCOPE

1

SCOPE

3D Tiles is designed for streaming and rendering massive 3D geospatial content such as Photogrammetry, 3D Buildings, BIM/CAD, Instanced Features, Point Clouds, Gaussian Splats, Voxels, and Vector Data. It defines a hierarchical data structure built on glTF which delivers renderable content. 3D Tiles does not define explicit rules for visualization of the content; a client may visualize 3D Tiles data however it sees fit.



2

CONFORMANCE

Annex A of this document contains the 3DTILES_tileset glTF 2.1 extension required to implement 3D Tiles.

All figures, examples, notes, and background information are informative.

2.1. Conformance Class: Core

A conforming 3D Tiles 2.0 client or server **SHALL** implement the core requirements of the specification.

2.1.1. Requirement 1: glTF 2.1

A conforming implementation **SHALL** support the glTF 2.1 specification.

2.1.2. Requirement 2: 3DTILES_tileset

A conforming implementation **SHALL** support the 3DTILES_tileset glTF extension.

2.2. Conformance Class: Coordinate Reference System

A conforming implementation **SHALL** implement the coordinate reference system requirements of the specification.

2.2.1. Requirement 1: Support of both Locally-Referenced and Globally-Referenced Tilesets

A conforming implementation **MAY** support both locally-referenced and globally-referenced tilesets.

2.2.2. Requirement 2: Support Locally-Referenced Tilesets

A conforming locally-referenced tileset **SHALL** support use of a local coordinate reference system. Tilesets using a local coordinate reference system **SHALL NOT** use the EXT_geospatial_crs or EXT_georeference glTF Extensions.

2.2.3. Requirement 3: Support Globally-Referenced Tilesets

A conforming globally-referenced tileset **SHOULD** support use of a global coordinate reference system. Tilesets using a global coordinate reference system **SHALL** use the EXT_geospatial_crs glTF Extension.

2.3. Conformance Class: Implicit Tiling

A conforming implementation **SHOULD** implement the implicit tiling requirements of the specification.

2.3.1. Requirement 1: Support 3DTILES_implicit_tiling

A conforming implementation that supports implicit tiling **SHALL** support the 3DTILES_implicit_tiling glTF extension.

2.3.2. Requirement 2: Support 3DTILES_subtree

A conforming implementation that supports implicit tiling **SHALL** support the 3DTILES_subtree glTF extension.

2.4. Conformance Class: 3D Gaussian Splatting Tiles Content Type

A conforming implementation **MAY** implement the 3D Gaussian Splatting tiles content type requirements of the specification.

2.4.1. Requirement 1: Support KHR_gaussian_splatting

A conforming implementation that supports 3D Gaussian Splatting tile content type **SHALL** support the KHR_gaussian_splatting glTF extension.

2.4.2. Requirement 2: Support KHR_gaussian_splatting_compression_spz_2

A conforming implementation that supports the 3D Gaussian Splatting tile content type **SHALL** support the KHR_gaussian_splatting_compression_spz_2 glTF extension.

2.5. Conformance Class: Architecture, Engineering, and Construction Content Type

A conforming implementation **MAY** implement the Architecture, Engineering, and Construction content type requirements of the specification.

2.5.1. Requirement 1: Support KHR_mesh_primitive_restart

A conforming implementation that supports the Architecture, Engineering, and Construction content type **MAY** support the KHR_mesh_primitive_restart glTF extension.

2.5.2. Requirement 2: Support EXT_mesh_primitive_edge_visibility

A conforming implementation that supports the Architecture, Engineering, and Construction content type **MAY** support the EXT_mesh_primitive_edge_visibility glTF extension.

2.5.3. Requirement 3: Support EXT_textureInfo_constant_lod

A conforming implementation that supports the Architecture, Engineering, and Construction content type **MAY** support the EXT_textureInfo_constant_lod glTF extension.

2.5.4. Requirement 4: Support BENTLEY_materials_line_style

A conforming implementation that supports the Architecture, Engineering, and Construction content type **MAY** support the BENTLEY_materials_line_style glTF extension.

2.5.5. Requirement 5: Support BENTLEY_materials_point_style

A conforming implementation that supports the Architecture, Engineering, and Construction content type **MAY** support the BENTLEY_materials_point_style glTF extension.

2.5.6. Requirement 6: Support BENTLEY_materials_planar_fill

A conforming implementation that supports the Architecture, Engineering, and Construction content type **MAY** support the BENTLEY_materials_planar_fill glTF extension.

2.6. Conformance Class: Time-dynamic Tiles Content Type

A conforming implementation **MAY** implement the vector tiles content type requirements of the specification.

2.6.1. Requirement 1: Support KHR_node_visibility

A conforming implementation that supports the time-dynamic tile content type **SHALL** support the KHR_node_visibility glTF extension.

2.6.2. Requirement 2: Support EXT_node_visibility_conditions

A conforming implementation that supports the time-dynamic tile content type **SHALL** support the EXT_node_visibility_conditions glTF extension.

2.7. Conformance Class: Vector Tiles Content Type

A conforming implementation **MAY** implement the vector tiles content type requirements of the specification.

2.7.1. Requirement 1: Support 3DTILES_tileset_vectors

A conforming implementation that supports the vector tile content type **SHALL** support the 3DTILES_tileset_vectors glTF extension.

2.7.2. Requirement 2: Support KHR_mesh_primitive_restart

A conforming implementation that supports the vector tile content type **SHALL** support the KHR_mesh_primitive_restart glTF extension.

2.7.3. Requirement 3: Support EXT_mesh_polygon

A conforming implementation that supports the vector tile content type **SHOULD** support the EXT_mesh_polygon glTF extension.

2.7.4. Requirement 4: Support 3DTILES_layers

A conforming implementation that supports the vector tile content type **MAY** support the 3DTILES_layers glTF extension.

2.8. Conformance Class: Voxel Tiles Content Type

A conforming implementation **MAY** implement the voxel tiles content type requirements of the specification.

2.8.1. Requirement 1: Support 3DTILES_implicit_tiling

A conforming implementation that supports voxel tile content type **SHALL** support the 3DTILES_implicit_tiling glTF extension.

2.8.2. Requirement 2: Support 3DTILES_subtree

A conforming implementation that supports voxel tile content type **SHALL** support the 3DTILES_subtree glTF extension.

2.8.3. Requirement 3: Support 3DTILES_tileset_voxels

A conforming implementation that supports voxel tile content type **SHALL** support the 3DTILES_tileset_voxels glTF extension.

2.8.4. Requirement 4: Support EXT_voxels

A conforming implementation that supports voxel tile content type **SHALL** support the EXT_voxels glTF extension.

2.8.5. Requirement 5: Support 3DTILES_shape_ellipsoid_region

A conforming implementation that supports voxel tile content type **SHOULD** support the 3DTILES_shape_ellipsoid_region glTF extension.

2.8.6. Requirement 6: Support 3DTILES_shape_cylinder_region

A conforming implementation that supports voxel tile content type **SHOULD** support the 3DTILES_shape_cylinder_region glTF extension.



3

NORMATIVE REFERENCES

The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

EPSG: 4978, 2020. https://epsg.org/crs_4978/WGS-84.html

EPSG: 4979, 2020. https://epsg.org/crs_4979/WGS-84.html

Masinter L: IETF RFC 2397, *The “data” URL scheme*. RFC Publisher (1998). <https://www.rfc-editor.org/info/rfc2397>.

Yergeau F: IETF RFC 3629, *UTF-8, a transformation format of ISO 10646*. RFC Publisher (2003). <https://www.rfc-editor.org/info/rfc3629>.

Berners-Lee T, Fielding R, Masinter L: IETF RFC 3986, *Uniform Resource Identifier (URI): Generic Syntax*. RFC Publisher (2005). <https://www.rfc-editor.org/info/rfc3986>.

Internet Engineering Task Force (committee): IETF RFC 8259, *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC Publisher (2017). <https://www.rfc-editor.org/info/rfc8259>.

Andreas Plesch and Mike McCann. 2015. The X3D geospatial component: X3DOM implementation of GeoOrigin, GeoLocation, GeoViewpoint, and GeoPositionInterpolator nodes. In *Proceedings of the 20th International Conference on 3D Web Technology (Web3D '15)*. Association for Computing Machinery, New York, NY, USA, 31–37. <https://doi.org/10.1145/2775292.2775315>

Khronos Group: glTF 2.1, 2026. <https://github.com/KhronosGroup/glTF/issues/2585>

Khronos Group: KHR_mesh_primitive_restart, 2026. https://github.com/CesiumGS/glTF/tree/donmccurdy/KHR_mesh_primitive_restart/extensions/2.0/Khronos/KHR_mesh_primitive_restart/README.md

Khronos Group: KHR_meshopt_compression, 2026. https://github.com/KhronosGroup/glTF/tree/main/extensions/2.0/Khronos/KHR_meshopt_compression/README.md

Khronos Group: KHR_node_visibility, 2026. https://github.com/KhronosGroup/glTF/tree/main/extensions/2.0/Khronos/KHR_node_visibility/README.md

Khronos Group: KHR_gaussian_splatting, 2026. https://github.com/KhronosGroup/glTF/tree/main/extensions/2.0/Khronos/KHR_gaussian_splatting/README.md

Khronos Group: KHR_gaussian_splatting_compression_spz_2, 2026. https://github.com/CesiumGS/glTF/tree/draft-splat-spz-split/extensions/2.0/Khronos/KHR_gaussian_splatting_compression_spz_2/README.md

Roger Lott, Open Geospatial Consortium: OGC 18-010r7, *Geographic information – Well-known text representation of coordinate reference systems*. Open Geospatial Consortium (2019). <http://www.opengis.net/doc/is/wkt-crs/2.0.6>.

W3C: W3C css-color-3, *CSS Color Module Level 3*. World Wide Web Consortium <https://www.w3.org/TR/css-color-3/>.

3D Tiles: EXT_node_visibility_conditions, 2026. https://github.com/CesiumGS/glTF/tree/3d-tiles-2.0/extensions/2.1/Vendor/EXT_node_visibility_conditions/README.md

3D Tiles: EXT_node_visibility_volume, 2026. https://github.com/CesiumGS/glTF/tree/3d-tiles-2.0/extensions/2.1/Vendor/EXT_node_visibility_volume

3D Tiles: EXT_geospatial_crs, 2026. https://github.com/CesiumGS/glTF/tree/3d-tiles-2.0/extensions/2.1/Vendor/EXT_geospatial_crs/README.md

3D Tiles: EXT_geospatial_crs_wkid, 2026. https://github.com/CesiumGS/glTF/tree/3d-tiles-2.0/extensions/2.1/Vendor/EXT_geospatial_crs_wkid/README.md

3D Tiles: EXT_geospatial_crs_wkt2, 2026. https://github.com/CesiumGS/glTF/tree/3d-tiles-2.0/extensions/2.1/Vendor/EXT_geospatial_crs_wkt2/README.md

3D Tiles: EXT_georeference, 2026. https://github.com/CesiumGS/glTF/tree/3d-tiles-2.0/extensions/2.1/Vendor/EXT_georeference/README.md

3D Tiles: EXT_mesh_polygon, 2026. https://github.com/CesiumGS/glTF/blob/donmccurdy/EXT_mesh_polygon/extensions/2.0/Vendor/EXT_mesh_polygon/README.md

3D Tiles: EXT_structural_metadata, 2026. https://github.com/CesiumGS/glTF/tree/3d-tiles-2.0/extensions/2.0/Vendor/EXT_structural_metadata/README.md

3D Tiles: EXT_mesh_features, 2026. https://github.com/CesiumGS/glTF/tree/3d-tiles-2.0/extensions/2.0/Vendor/EXT_mesh_features/README.md

3D Tiles: EXT_instance_features, 2026. https://github.com/CesiumGS/glTF/tree/3d-tiles-2.0/extensions/2.0/Vendor/EXT_instance_features/README.md

3D Tiles: EXT_voxels, 2026. https://github.com/CesiumGS/glTF/tree/3d-tiles-2.0/extensions/2.1/Vendor/EXT_voxels/README.md

3D Tiles: EXT_mesh_primitive_edge_visibility, 2026. https://github.com/KhronosGroup/glTF/tree/main/extensions/2.0/Vendor/EXT_mesh_primitive_edge_visibility/README.md

3D Tiles: EXT_textureInfo_constant_lod, 2026. https://github.com/CesiumGS/glTF/tree/vendor-extensions/extensions/2.0/Vendor/EXT_textureInfo_constant_lod/README.md

3D Tiles: BENTLEY_materials_planar_fill, 2026. https://github.com/CesiumGS/glTF/tree/vendor-extensions/extensions/2.0/Vendor/BENTLEY_materials_planar_fill/README.md

3D Tiles: BENTLEY_materials_line_style, 2026. https://github.com/CesiumGS/glTF/tree/vendor-extensions/extensions/2.0/Vendor/BENTLEY_materials_line_style/README.md

3D Tiles: BENTLEY_materials_point_style, 2026. https://github.com/CesiumGS/glTF/tree/vendor-extensions/extensions/2.0/Vendor/BENTLEY_materials_point_style/README.md

The background features a dark blue field with several thin, light yellow lines intersecting at various points. Three of these intersection points are marked with small yellow dots. The lines create a network of triangles and other geometric shapes across the page.

4

TERMS AND DEFINITIONS

TERMS AND DEFINITIONS

This document uses the terms defined in [OGC Policy Directive 49](#), which is based on the ISO/IEC Directives, Part 2, Rules for the structure and drafting of International Standards. In particular, the word “shall” (not “must”) is the verb form used to indicate a requirement to be strictly followed to conform to this document and OGC documents do not use the equivalent phrases in the ISO/IEC Directives, Part 2.

This document also uses terms defined in the OGC Standard for Modular specifications (OGC 08-131r3), also known as the ‘ModSpec’. The definitions of terms such as standard, specification, requirement, and conformance test are provided in the ModSpec.

For the purposes of this document, the following additional terms and definitions apply.

4.1. Availability

Data specifying which tiles/contents/child subtrees exist within a single subtree of an implicit tileset.

4.2. Bounding Volume

A closed volume completely containing the union of a set of geometric objects. See [Wikipedia: Bounding volume](#).

4.3. Child subtree

A subtree reachable from an available tile in the bottommost row of a subtree of an implicit tileset.

4.4. Entity

An entity, in the context of 3D Tiles metadata, is an instance of a metadata class, populated with property values conforming to a metadata class definition.

4.5. Feature

In 3D Tiles, an individual component of a tile, such as 3D models representing buildings or trees, or points in a point cloud. Each feature has position and appearance properties and additional application-specific properties.

4.6. Geometric Error

A nonnegative number that defines the error, in meters, introduced if this tile is rendered and its children are not. At runtime, the geometric error is used to compute Screen-Space Error (SSE).

4.7. glTF

An API-neutral runtime asset delivery format for 3D assets and scenes.

4.8. Hierarchical Level of Detail (HLOD)

Decreasing the complexity of a 3D representation according to metrics such as object importance or distance from the tile to the observation point or camera position. Generally, higher levels of detail provide greater visual fidelity. See [Wikipedia: Level of detail](#)

4.9. Implicit tiling

A description of a tileset using recursive subdivision. The structure and semantics of implicit tiling are defined by the `3DTILES_implicit_tiling` extension. See [3DTILES_implicit_tiling](#).

4.10. Implicit root tile

A tile that contains the `3DTILES_implicit_tiling` extension, and therefore denotes the root of an implicit tileset.

4.11. Metadata

In the context of 3D Tiles, metadata is equivalent to the concept of attribution in other Open Geospatial Consortium standards. Metadata refers to non-spatial information that is assigned to entities at multiple granularities, including tilesets, tiles, contents, and features.

4.12. Metadata class

The description of the structure of an entity that contains 3D Tiles metadata, consisting of multiple metadata properties.

4.13. Metadata property

An element of a metadata class that defines a name and a type for the corresponding element of a metadata entity.

4.14. Octree

A 3D subdivision scheme that divides each bounding volume into 8 smaller bounding volumes along the midpoint of the x, y, and z axes.

4.15. Quadtree

A 2D subdivision scheme that divides each bounding volume into 4 smaller bounding volumes along the midpoint of the x and y axes.

4.16. Schema

A set of metadata classes and enums that define the structure and type of metadata.

4.17. Screen-Space Error (SSE)

The error measured in pixels that determines if a tile is sufficiently detailed for the current view or if its children should be considered.

4.18. Spatial Coherence

The union of all content of the child tiles is completely inside the parent tile's bounding volume.

4.19. Style

A set of expressions to be evaluated which modify how each feature in a tileset is displayed.

4.20. Subtree

A fixed-size section of the tree of an implicit tileset that contains availability information. The structure and semantics of a subtree are defined by the 3DTILES_subtree extension. See [3DTILES_subtree](#).

4.21. Subtree file

A file storing information about a specific subtree of an implicit tileset.

4.22. Subdivision scheme

A recursive pattern of dividing a parent tile into smaller child tiles occupying the same area. This is done by uniformly dividing the bounding volume of the parent tile with an implicit tileset.

4.23. Template URI

A URI pattern containing tile coordinates for directly addressing tiles in an implicit tileset.

4.24. Tile

In 3D Tiles, a tile is a subset of a tileset containing a reference to renderable content and the metadata used to determine if the content is rendered. Tiles are defined using a glTF node within a 3DTILES_tileset extension property. See the Tile section in [3DTILES_tileset](#).

4.25. Content

The renderable content of a tile as a glTF asset.

4.26. Tileset

In 3D Tiles, a collection of 3D Tiles tile instances organized into a spatial data structure and additional metadata, such that the aggregation of these tiles represent some 3D content at various levels of detail. The structure and semantics of a tileset are defined by the 3DTILES_tileset extension. A valid tileset in 3D Tiles 2.0 MUST use the 3DTILES_tileset extension for glTF 2.1. See the Tileset section in 3DTILES_tileset.



5

CONVENTIONS



CONVENTIONS

No conventions are specified in this document.



6

RELATIONSHIP TO GLTF

A 3D Tiles 2.0 tileset is a glTF 2.1 asset. The 3DTILES_tileset extension is a normative part of the 3D Tiles 2.0 specification and defines the structure of the tileset. Specifically, the 3DTILES_tileset extension defines:

- The hierarchical structure of the tileset.
- The tile object, which represents a tile in the tileset hierarchy.
- The definition of a tile's content, including its URI.
- The geometric error and refinement strategy (ADD or REPLACE) for each tile.
- The mechanism for supporting implicit tiling.
- The mechanism for linking to external tilesets.
- The coordinate reference system of the tileset.
- The mechanism for assigning application-specific *metadata* using the 3D Metadata Specification.

All tile content is represented as glTF assets, either embedded or external.

Any feature that is normatively defined in the glTF 2.1 specification is considered a normative part of 3D Tiles 2.0. A conforming 3D Tiles 2.0 implementation **SHOULD** support all normative features of glTF 2.1 unless otherwise stated.



7

3D TILES FORMAT SPECIFICATION

7.1. Introduction

In 3D Tiles, a *tileset* is a set of *tiles* organized in a spatial data structure, the *tree*. Each tile may reference renderable *content*.

The content references a set of *features*, such as 3D models representing buildings or trees, or points in a point cloud. Each feature has position and appearance properties and additional application-specific properties. A client may choose to select features at runtime and retrieve their properties for visualization or analysis.

Tiles are organized in a tree which incorporates the concept of Hierarchical Level of Detail (HLOD) for optimal rendering of spatial data. Each tile has a *bounding volume*, an object defining a spatial extent completely enclosing its content. The tree has spatial coherence; the content for child tiles are completely inside the parent's bounding volume.

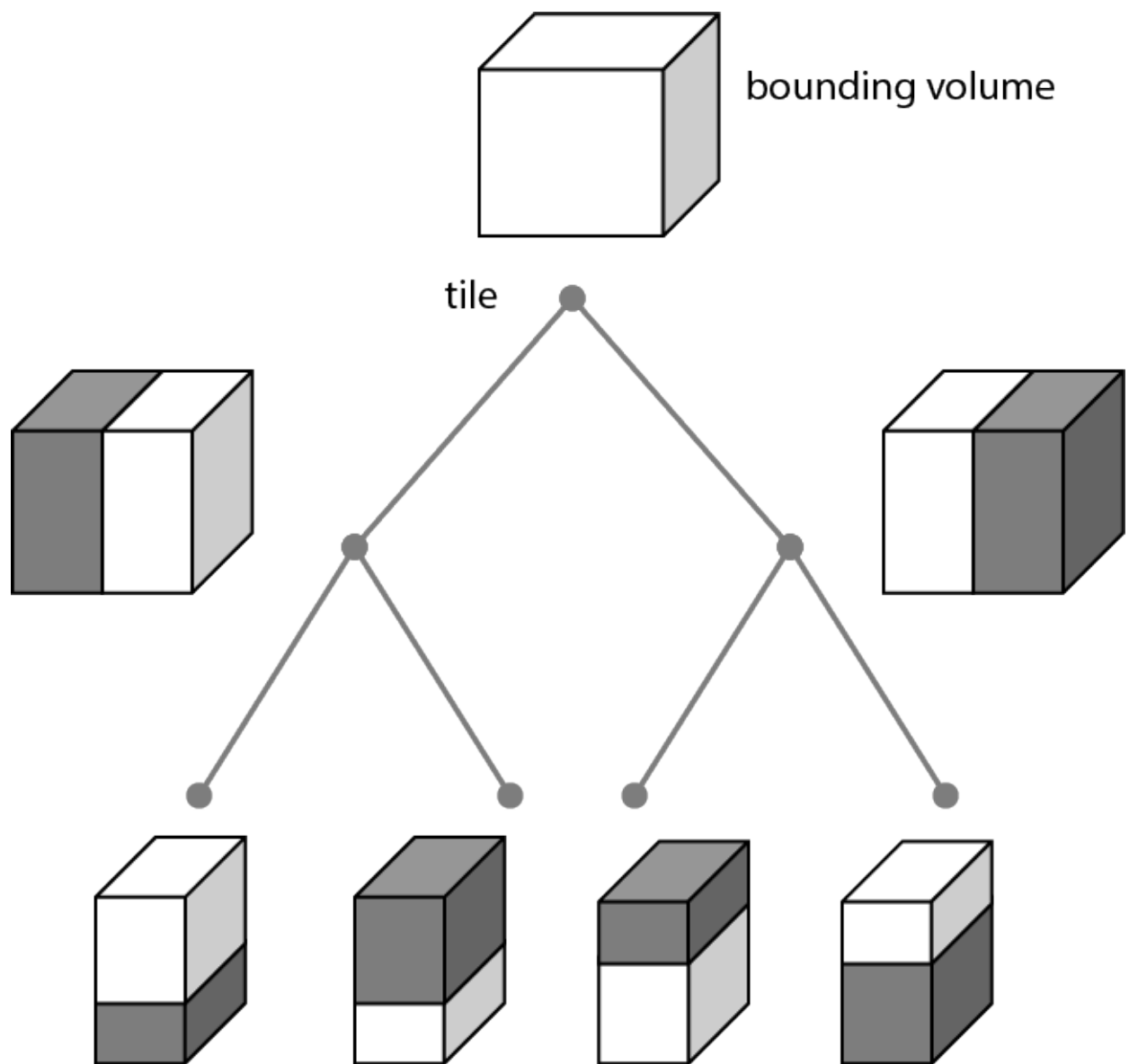


Figure 1 — A tree of tiles

A tileset may use a 2D spatial tiling scheme similar to raster and vector tiling schemes (like a Web Map Tile Service (WMTS) or XYZ scheme) that serve predefined tiles at several levels of detail (or zoom levels). However since the content of a tileset is often non-uniform or may not easily be organized in only two dimensions, the tree can be any spatial data structure with spatial coherence, including k-d trees, quadrees, octrees, and grids. Implicit tiling defines a concise representation of quadrees and octrees.

Application-specific *metadata* may be provided at multiple granularities within a tileset. Properties may be associated with high-level entities like tilesets, tiles, contents, or features, or with individual vertices and texels. Metadata conforms to a well-defined type system described by the 3D Metadata Specification, which may be extended with application- or domain-specific semantics.

3D Tiles supports a variety of content types and features through its alignment with glTF. This allows for rich, interactive 3D experiences. Some of these features include:

- Advanced rendering for architectural, engineering, and construction (AEC) models, including edge display, specialized line and point styles, and techniques to avoid visual artifacts for overlapping surfaces.
- Support for both 2D and 3D vector data using a 3D-native data model consistent with the rest of the standard and with glTF.
- Time-dynamic content, where a tileset can hold multiple representations for the same location. This enables visualizations over time, comparison of design revisions, or switching between different data types like meshes and point clouds.
- Support for volumetric data through voxels, optimized for streaming with support for sparse datasets and compression.
- Rendering of complex scenes using 3D Gaussian Splatting, which is fully compatible with the 3D Tiles level-of-detail hierarchy.

Optionally a 3D Tiles Style, or *style*, may be applied to a tileset. A style defines expressions to be evaluated which modify how each feature is displayed.

7.2. Alignment with glTF

glTF™ is a royalty-free specification for the efficient transmission and loading of 3D scenes and models by engines and applications. glTF minimizes the size of 3D assets, and the runtime processing needed to unpack and use them. glTF defines an extensible publishing format that streamlines authoring workflows by enabling the interoperable use of 3D content across the industry.¹

3D Tiles is built on glTF 2.1. As such, many core concepts are inherited directly from the glTF 2.1 specification or defined through glTF extensions. JSON structure, URIs, and Units are defined in the glTF 2.1 specification.

The 3DTILES_tileset extension is a normative part of the 3D Tiles 2.0 specification and defines the structure of the tileset.

7.2.1. File Extensions and Media Types

File extensions and media types are defined by the glTF 2.1 specification.

¹<https://www.khronos.org/glTF/>

The 3DTILES_tileset extension has specific requirements for file extensions. Assets using this extension SHOULD use the .tileset.gltf or .tileset.glb file extensions. The entry tileset SHOULD be named root.tileset.gltf. See the 3DTILES_tileset extension specification for more details. See the section titled File Extensions in the 3DTILES_tileset extension.

The 3DTILES_subtree extension, used for implicit tiling, also has specific file extension requirements. Subtree assets for implicit tiling will use the .subtree.gltf file extension. See the section titled [File Extensions in the 3DTILES_subtree extension](#).

7.2.2. glTF Extensions

As a 3D Tiles tileset must be a valid glTF 2.1 asset, any extension compatible with glTF 2.1 may be used with 3D Tiles 2.0. There are some specific extensions that are normatively defined by 3D Tiles 2.0. These extensions use the 3DTILES_ prefix to differentiate them. Definitions as well as informative descriptions to the extensions can be found in the glTF Extensions for 3D Tiles section and the 3DTILES_tileset extension.

7.3. Concepts

Core concepts are normatively defined within the 3DTILES_tileset glTF extension. Additional details can be found in the glTF Extensions for 3D Tiles section and the extension specification.

7.4. Declarative styling specification

3D Tiles includes concise declarative styling defined with JSON and expressions written in a small subset of JavaScript augmented for styling.

For complete details, see the Declarative Styling specification.

7.5. Compatibility with previous versions of 3D Tiles

3D Tiles 2.0 introduces changes that are not fully backwards compatible with previous versions of 3D Tiles. Tilesets created in 3D Tiles 1.1 and earlier are not valid 3D Tiles 2.0 tilesets. A full list of [breaking changes and migration notes](#) can be found on the 3D Tiles GitHub repository.



8

GLTF EXTENSIONS FOR 3D TILES

8.1. Tileset Extensions

As a 3D Tiles tileset must be a valid glTF 2.1 asset, any extension compatible with glTF 2.1 may be used with 3D Tiles 2.0. There are some specific extensions that are normatively defined by 3D Tiles 2.0. These extensions use the `3DTILES_` prefix to differentiate them.

8.1.1. Core Extension

The `3DTILES_tileset` extension specifies a well defined subset of glTF 2.1 for representing a tileset in [3D Tiles](#). It extends the node hierarchy to support Hierarchical Level of Detail (HLOD) for streaming massive 3D scenes. Additionally, it depends on core glTF 2.1 features like external assets and bounding volumes.

These concepts are normatively defined within the `3DTILES_tileset` glTF extension.

- 3D Tiles
- Tilesets
- Tiles
- Tile Content
- External Tilesets
- Geometric Error
- Refinement
- Bounding Volumes
- Transforms
- Spatial Coherence
- Spatial Data Structures
- Coordinate Reference Systems
- Implicit Tiling

Additional normative concepts are defined in both 3DTILES_tileset and several supporting extensions, including:

- Metadata
- Layers
- Horizon Occlusion Point
- Visibility Bounding Volumes
- Declarative Styling

8.2. Content Extensions

Any glTF extension compatible with glTF 2.1 can be used to extend content files. Many core content features of 3D Tiles 2.0 are powered entirely by glTF extensions. Supporting certain types of content means supporting these glTF extensions. This is not an exhaustive list—other extensions may exist within the glTF community that could be used within 3D Tiles 2.0.

8.2.1. 3D Gaussian Splatting

3D Tiles 2.0 supports 3D Gaussian Splatting content via the [KHR_gaussian_splatting](#) glTF extension. Implementations supporting [KHR_gaussian_splatting](#) should also support [KHR_gaussian_splatting_compression_spz_2](#), which defines compression of splat data using the SPZ format. Uncompressed splat content may be used where compression is not required. 3D Gaussian splatting tilesets are compatible with the hierarchical level-of-detail mechanism of 3D Tiles, such that clients may stream and cull splat content using the same traversal rules as other tile content types.

See these glTF extension definitions for details:

- [KHR_gaussian_splatting](#)
- [KHR_gaussian_splatting_compression_spz_2](#)

8.2.2. Architectural, Engineering, and Construction Rendering

3D Tiles 2.0 supports edge rendering for AEC workflows through the glTF extensions [KHR_mesh_primitive_restart](#) and [EXT_mesh_primitive_edge_visibility](#). Implementations for AEC workflows may recognize both extensions in tile content and can either render the content according to the extension definitions or report the extension as unsupported in accordance with glTF 2.1.

For the styling of points and lines in CAD content, 3D Tiles 2.0 supports the [BENTLEY_materials_line_style](#) and [BENTLEY_materials_point_style](#) glTF extensions. These extensions define the properties used to represent hidden and conceptual elements common in civil and engineering rendering workflows.

Implementations may use [EXT_textureInfo_constant_lod](#) to maintain consistent texture pattern scale across zoom levels. Implementations may also use [BENTLEY_materials_planar_fill](#) to mitigate z-fighting between co-located surfaces such as labels, roads, and overlapping infrastructure.

See these glTF extension definitions for details:

- [KHR_mesh_primitive_restart](#)
- [EXT_mesh_primitive_edge_visibility](#)
- [BENTLEY_materials_line_style](#)
- [BENTLEY_materials_point_style](#)
- [EXT_textureInfo_constant_lod](#)
- [BENTLEY_materials_planar_fill](#)

8.2.3. Time-dynamic Content

Using the [EXT_node_visibility_conditions](#) extension, implementations may support time-dynamic 3D tiles, enabling multiple representations of the same spatial region within a single tileset (for example, temporal states of terrain, design-model revisions, or alternative geometries such as point clouds and meshes).

This extension builds upon the the [KHR_node_visibility](#) extension by adding support for visibility conditions. Any tile may be tagged with one or more attribute key/value pairs. Tilesets are not required to tag every tile; untagged tiles are treated as unconditional. Clients select tile content by evaluating these tags against a selection expression supplied at runtime. Where no selection expression is supplied, clients can render the default content declared by the tileset author.

Authoring tools may use time-dynamic 3D tiles to publish incremental updates without rewriting an entire tileset.

See these glTF extension definitions for details:

- [EXT_node_visibility_conditions](#)
- [KHR_node_visibility](#)

8.2.4. Vector Tiles

3D Tiles 2.0 supports both 2D and 3D vector data using a 3D-native data model consistent with the rest of the standard and with glTF 2.1. Relevant dependencies include [3DTILES_tileset_vectors](#), [KHR_mesh_primitive_restart](#) and [EXT_mesh_polygon](#). Tilesets can represent road centerlines, building footprints, large-scale map features, and AEC design linework as vector data. Payloads may be uncompressed or compressed using the [KHR_meshopt_compression](#) extension.

3D Tiles 2.0 is designed for efficient transmission and rendering of 3D vector data, which differentiates it from established vector formats like Mapbox Vector Tiles and GeoJSON. Implementations may provide interoperability paths to and from those formats where appropriate. Styling of vector content uses the same declarative styling language defined for other 3D Tiles content types.

See these glTF extension definitions for details:

- [3DTILES_tileset_vectors](#)
- [KHR_mesh_primitive_restart](#)
- [EXT_mesh_polygon](#)
- [KHR_meshopt_compression](#)

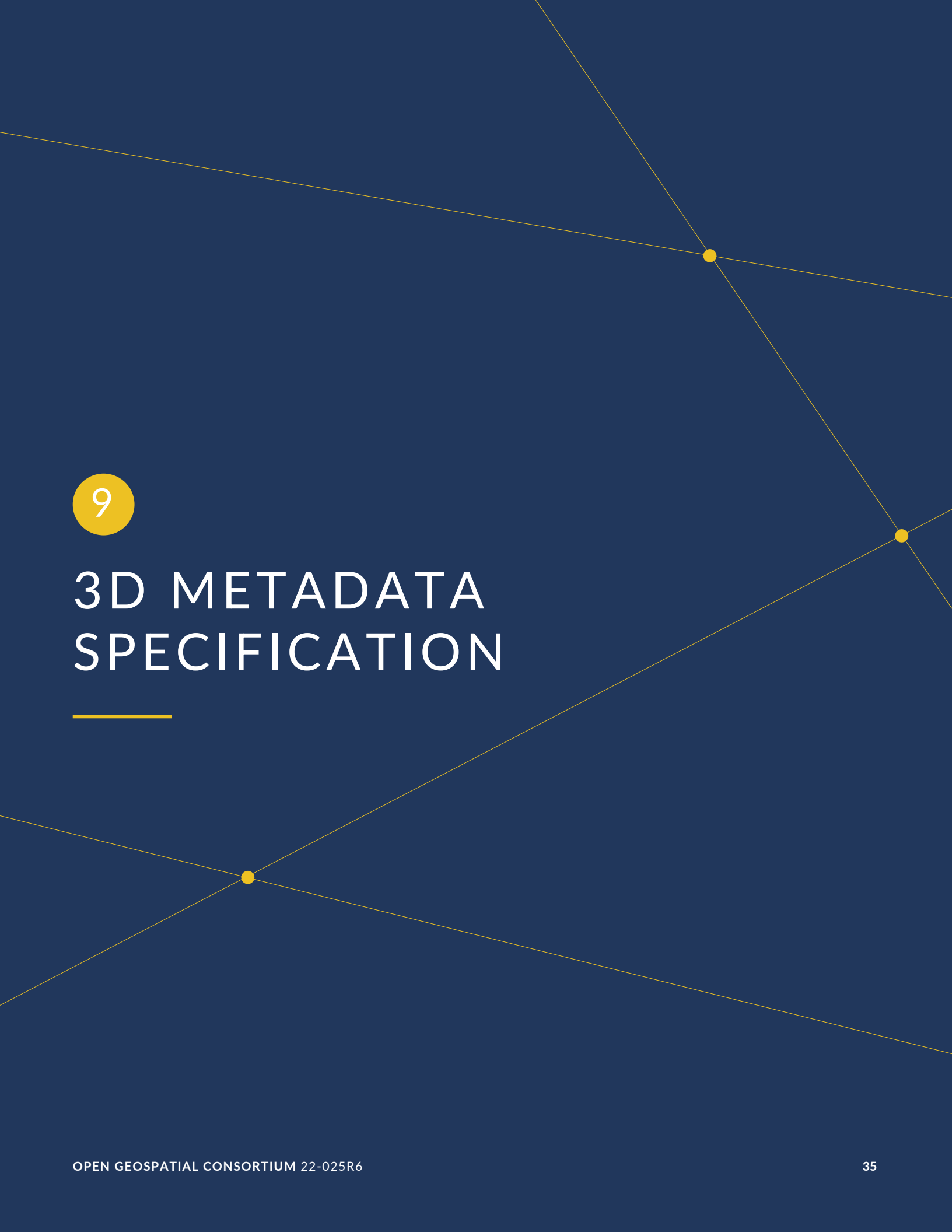
8.2.5. Voxels

3D Tiles 2.0 supports voxel volumetric datasets. Relevant dependencies include the [3DTILES_tileset_voxels](#), [EXT_voxels](#), [3DTILES_shape_cylinder_region](#), and [3DTILES_shape_ellipsoid_region](#) glTF 2.1 extensions in combination with the implicit tiling mechanism defined in the [3DTILES_tileset](#) extension.

Both dense and sparse voxel volumes can be supported. Voxel payloads can be uncompressed, or can be compressed using gzip, the glTF [KHR_meshopt_compression](#) extension, or a combination of the available methods.

See these glTF extension definitions for details:

- [3DTILES_tileset_voxels](#)
- [EXT_voxels](#)
- [3DTILES_shape_cylinder_region](#)
- [3DTILES_shape_ellipsoid_region](#)
- [KHR_meshopt_compression](#)



9

3D METADATA SPECIFICATION

9.1. Overview

The 3D Metadata Specification defines a standard format for structured metadata in 3D content. Metadata — represented as entities and properties — may be closely associated with parts of 3D content, with data representations appropriate for large, distributed datasets. For the most detailed use cases, properties allow vertex- and texel-level associations; higher-level property associations are also supported.

Many domains benefit from structured metadata — typical examples include historical details of buildings in a city, names of components in a CAD model, descriptions of regions on textured surfaces, and classification codes for point clouds.

The specification defines core concepts to be used by multiple 3D formats, and is language and format agnostic. This document defines concepts with purpose and terminology, but does not impose a particular schema or serialization format for implementation. For use of the format outside of abstract conceptual definitions, see:

- [EXT_structural_metadata](#) — Assigns metadata to tileset, tile, content, vertices, texels, and features in a glTF asset.

The specification does not enumerate or define the semantic meanings of metadata, and assumes that separate specifications will define semantics for their particular application or domain. Identifiers for externally-defined semantics can be stored within the 3D Metadata Specification.

9.2. Concepts

This specification defines metadata schemas and methods for encoding metadata.

Schemas contain a set of **classes** and **enums**. A class represents a category of similar entities, and is defined as a set of **properties**. Each property describes values of a particular type. An enum defines a set of named values representing a single value type, and may be referenced by class properties. Schema definitions do not describe how entities or properties are stored, and may be represented in a file format in various ways. Schemas can be reused across multiple assets or even file formats.

Entities are instantiations of a class, populated with **property values** conforming to the class definition. Every property value of an entity shall be defined by its class, and an entity shall not have extraneous property values. Properties of a class may be required, in which case all entities instantiating the class are required to include them.

NOTE: Entities may be defined at various levels of abstraction. Within a large dataset, individual vertices or texels may represent entities with granular metadata properties. Vertices and texels may be organized into higher-order groups (such as meshes, scene graphs, or tilesets) having their own associated properties.

Metadata, as used throughout this specification, refers to any association of 3D content with entities and properties, such that entities represent meaningful units within an overall structure. Other common definitions of metadata, particularly in relation to filesystems or networking as opposed to 3D content, remain outside the scope of the document.

Property values are stored with flexible representations to allow compact transmission and efficient lookups. This specification defines two possible storage formats.

9.2.1. Identifiers

Throughout this specification, IDs (identifiers) are strings that match the regular expression `^[a-zA-Z_][a-zA-Z0-9_]*$`: Strings that consist of upper- or lowercase letters, digits, or underscores, starting with either a letter or an underscore. These strings should be camel case strings that are human-readable (wherever possible). When IDs subject to these restrictions are not sufficiently clear for human readers, applications should also provide a name for the structures that support dedicated names.

9.3. Schemas

9.3.1. Schema

A schema defines the organization and types of metadata used in 3D content, represented as a set of classes and enums. Class definitions are referenced by entities whose metadata conforms to the class definition. This provides a consistent and machine-readable structure for all entities in a dataset.

Components of a schema are listed below, and implementations may define additional components.

9.3.1.1. ID

IDs (`id`) are unique identifiers for a schema.

NOTE: The schema ID is *required* for each schema. The main purpose of this ID is to be able to resolve possible ambiguities. The exact mechanism for this disambiguation depends on the client application, and how the access to metadata properties is implemented. But the schema ID makes sure that a compound identifier of the form <schema-id>.<class-name>.<property-name> is globally unique, even when the same class name and property name appears in multiple schemas.

9.3.1.2. Version

Schema version (*version*) is an application-specific identifier for a given schema revision. Version shall be a string, and should be syntactically compatible with [SemVer](#).

When a schema has multiple versions, the (*id*, *version*) pair uniquely identifies a particular schema and revision.

NOTE: Valid semantic versions include strings like 0.1.2, 1.2.3, and 1.2.3-alpha.

9.3.1.3. Name

Names (*name*) provide a human-readable label for a schema, and are not required to be unique. Names shall be valid Unicode strings, and should be written in natural language.

9.3.1.4. Description

Descriptions (*description*) provide a human-readable explanation of a schema, its purpose, or its contents. Typically at least a phrase, and possibly several sentences or paragraphs. Descriptions shall be valid Unicode strings.

9.3.1.5. Enums

Unordered set of enums.

9.3.1.6. Classes

Unordered set of classes.

9.3.2. Enum

An enum consists of a set of named values, represented as (*string*, *integer*) pairs. Each enum is identified by a unique ID.

NOTE: A “species” enum with three possible tree species, as well as an “Unknown” value.

- **ID:** “species”
- **Name:** “Species”
- **Description:** “Common tree species identified in the study.”
- **Value type:** INT32

Names and values of an example enum

NAME	VALUE
"Oak"	0
"Pine"	1
"Maple"	2
"Unknown"	-1

9.3.2.1. ID

IDs (id) are unique identifiers for an enum within a schema.

9.3.2.2. Name

Names (name) provide a human-readable label for an enum, and are not required to be unique within a schema. Names shall be valid Unicode strings, and should be written in natural language.

9.3.2.3. Description

Descriptions (description) provide a human-readable explanation of an enum, its purpose, or its contents. Typically at least a phrase, and possibly several sentences or paragraphs. Descriptions shall be valid Unicode strings.

9.3.2.4. Values

An enum consists of a set of named values, represented as (string, integer) pairs. The following enum value types are supported: INT8, UINT8, INT16, UINT16, INT32, UINT32, INT64, and UINT64. See the Component Type section for definitions of each. Smaller enum types limit

the range of possible enum values, and allow more efficient binary encoding. Duplicate names or values within the same enum are not allowed.

9.3.3. Class

Classes represent categories of similar entities, and are defined by a collection of one or more properties shared by the entities of a class. Each class has a unique ID within the schema, and each property has a unique ID within the class, to be used for references within the schema and externally.

9.3.3.1. ID

IDs (`id`) are unique identifiers for a class within a schema.

9.3.3.2. Name

Names (`name`) provide a human-readable label for a class, and are not required to be unique within a schema. Names shall be valid Unicode strings, and should be written in natural language.

9.3.3.3. Description

Descriptions (`description`) provide a human-readable explanation of a class, its purpose, or its contents. Typically at least a phrase, and possibly several sentences or paragraphs. Descriptions shall be valid Unicode strings.

9.3.3.4. Properties

Unordered set of properties.

9.3.4. Property

9.3.4.1. Overview

Properties describe the type and structure of values that may be associated with entities of a class. Entities may omit values for a property, unless the property is required. Entities shall not contain values other than those defined by the properties of their class.

NOTE: The following example shows the basics of how classes describe the types of metadata. A `building` class describes the heights of various buildings in a dataset. Likewise, the `tree` class describes trees that have a height, species, and leaf color.

building

Properties and types of a building

PROPERTY	TYPE	COMPONENTTYPE
height	SCALAR	FLOAT32

tree

Properties and types of a tree

PROPERTY	TYPE	COMPONENTTYPE	ENUMTYPE
height	SCALAR	FLOAT32	
species	ENUM		species
leafColor	STRING		

9.3.4.2. ID

IDs (`id`) are unique identifiers for a property within a class.

9.3.4.3. Name

Names (`name`) provide a human-readable label for a property, and shall be unique to a property within a class. Names shall be valid Unicode strings, and should be written in natural language. Property names do not have inherent meaning; to provide such a meaning, a property shall also define a semantic.

NOTE: A typical ID / Name pair, in English, would be `localTemperature` and "Local Temperature". In Japanese, the name might be represented as “きおん”. Because IDs are restricted to identifiers, use of helpful property names is essential for clarity in many languages.

9.3.4.4. Description

Descriptions (`description`) provide a human-readable explanation of a property, its purpose, or its contents. Typically at least a phrase, and possibly several sentences or paragraphs. Descriptions shall be valid Unicode strings. To provide a machine-readable semantic meaning, a property shall also define a semantic.

9.3.4.5. Semantic

Property IDs, names, and descriptions do not have an inherent meaning. To provide a machine-readable meaning, properties may be assigned a semantic identifier string (`semantic`), indicating how the property's content should be interpreted. Semantic identifiers may be defined by external semantic references, and may be application-specific. Identifiers should be uppercase, with underscores as word separators.

NOTE: Semantic definitions might include temperature in degrees Celsius (e.g. `TEMPERATURE_DEGREES_CELSIUS`), time in milliseconds (e.g. `TIME_MILLISECONDS`), or mean squared error (e.g. `MEAN_SQUARED_ERROR`). These examples are only illustrative.

9.3.4.6. Type

A property's type (`type`) describes the structure of the value given for each entity.

Table 4 — Types of metadata properties

NAME	TYPE
SCALAR	Single numeric component
VEC2	Fixed-length vector with two (2) numeric components
VEC3	Fixed-length vector with three (3) numeric components
VEC4	Fixed-length vector with four (4) numeric components
MAT2	2×2 matrix with numeric components
MAT3	3×3 matrix with numeric components
MAT4	4×4 matrix with numeric components
STRING	A sequence of characters

NAME	TYPE
BOOLEAN	True or false
ENUM	An enumerated type

9.3.4.7. Component Type

Scalar, vector, and matrix types comprise of numeric components. Each component is an instance of the property's component type (`componentType`), with the following component types supported:

Table 5 — Component types of metadata properties

NAME	COMPONENTTYPE
INT8	Signed integer in the range [−128, 127]
UINT8	Unsigned integer in the range [0, 255]
INT16	Signed integer in the range [−32768, 32767]
UINT16	Unsigned integer in the range [0, 65535]
INT32	Signed integer in the range [−2147483648, 2147483647]
UINT32	Unsigned integer in the range [0, 4294967295]
INT64	Signed integer in the range [−9223372036854775808, 9223372036854775807]
UINT64	Unsigned integer in the range [0, 18446744073709551615]
FLOAT32	A number that can be represented as a 32-bit IEEE floating point number
FLOAT64	A number that can be represented as a 64-bit IEEE floating point number

Floating-point properties (FLOAT32 and FLOAT64) shall not include values NaN, +Infinity, or −Infinity.

NOTE: Developers of authoring tools should be aware that many JSON implementations support only numeric values that can be represented as IEEE-754 double precision floating point numbers. Floating point numbers should be representable as double precision IEEE-754 floats when encoded in JSON. When those numbers represent property values (such as `noData`, `min`, or `max`) having lower precision (e.g. single-precision float, 8-bit integer, or 16-bit integer), the

values should be rounded to the same precision in JSON to avoid any potential mismatches. Numeric property values encoded in binary storage are unaffected by these limitations of JSON implementations.

9.3.4.8. Enum Type

Enum properties are denoted by `ENUM`. An enum property shall additionally provide the ID of the specific enum it uses, referred to as its enum type (`enumType`).

9.3.4.9. Arrays

A property can be declared to be a fixed- and variable-length array, consisting of elements of the given type. For fixed-length arrays, a count (`count`) denotes the number of elements in each array, and shall be greater than or equal to 2. Variable-length arrays do not define a count and may have any length, including zero.

9.3.4.10. Normalized Values

Normalized properties (`normalized`) provide a compact alternative to larger floating-point types. Normalized values are stored as integers, but when accessed are transformed to floating-point according to the following equations:

Table 6 — Conversion of component types

COMPONENT INT TO FLOAT		FLOAT TO INT
INT8	$f = \max(i / 127.0, -1.0)$	$i = \text{round}(f * 127.0)$
UINT8	$f = i / 255.0$	$i = \text{round}(f * 255.0)$
INT16	$f = \max(i / 32767.0, -1.0)$	$i = \text{round}(f * 32767.0)$
UINT16	$f = i / 65535.0$	$i = \text{round}(f * 65535.0)$
INT32	$f = \max(i / 2147483647.0, -1.0)$	$i = \text{round}(f * 2147483647.0)$
UINT32	$f = i / 4294967295.0$	$i = \text{round}(f * 4294967295.0)$
INT64	$f = \max(i / 9223372036854775807.0, -1.0)$	$i = \text{round}(f * 9223372036854775807.0)$
UINT64	$f = i / 18446744073709551615.0$	$i = \text{round}(f * 18446744073709551615.0)$

`normalized` is only applicable to scalar, vector, and matrix types with integer component types.

NOTE: Depending on the implementation and the chosen integer type, there may be some loss of precision in values after denormalization. For example, if the implementation uses 32-bit floating point variables to represent the value of a normalized 32-bit integer, there are only 23 bits in the mantissa of the float, and lower bits will be truncated by denormalization. Client implementations should use higher precision floats when appropriate for correctly representing the result.

9.3.4.11. Offset and Scale

A property may declare an offset (`offset`) and scale (`scale`) to apply to property values. This is useful when mapping property values to a different range.

The `offset` and `scale` can be defined for types that either have a floating-point `componentType`, or when `normalized` is set to `true`. This applies to `SCALAR`, `VECN`, and `MATN` types, and to fixed-length arrays of these types. The structure of `offset` and `scale` is explained in the Property Values Structure section.

The following equation is used to transform the original property value into the actual value that is used by the client:

$$\text{transformedValue} = \text{offset} + \text{scale} * \text{normalize}(\text{value})$$

These operations are applied component-wise, both for array elements and for vector and matrix components.

The transformation that is described here allows arbitrary source value ranges to be mapped to arbitrary target value ranges, by first computing the float value for the original normalized value, and then mapping that floating point range to the desired target range.

NOTE: The result of transforming a normalized integer value into a floating point value may be lossy, as described in the section about Normalized Values. Depending on the range of property values, the values of `offset` and `scale`, and the floating point precision that is used in the client implementation, the computation may cause low-significance bits to be truncated from the final result. Client implementations should retain as much precision as reasonably possible.

When the `offset` for a property is not given, then it is assumed to be 0 for each component of the respective type. When the `scale` value of a property is not given, then it is assumed to be 1 for each component of the respective type. *Instances* of the class that defines the respective property can override the `offset`- and `scale` factors, to account for the actual range of property values that are provided by the instance.

9.3.4.12. Minimum and Maximum Values

Properties may specify a minimum (`min`) and maximum (`max`) value. Minimum and maximum values represent component-wise bounds of the valid range of values for a property. Both values are *inclusive*, meaning that they denote the smallest and largest allowed value, respectively.

The `min` and `max` value can be defined for `SCALAR`, `VECN`, and `MATN` types with numeric component types, and for fixed-length arrays of these types. The structure of `min` and `max` is explained in the Property Values Structure section.

For properties that are normalized, the component type of `min` and `max` is a floating point type. Their values represent the bounds of the final, transformed property values. This includes the normalization and offset- or scale computations, as well as other transforms or constraints that are not part of the class definition itself: A normalized unsigned value is in the range `[0.0, 1.0]` after the normalization has been applied, but `[min, max]` may specify a different value range.

For all other properties, the component type of `min` and `max` matches the `componentType` of the property, and the values are the bounds of the original property values.

NOTE: A property storing GPS coordinates might define a range of `[-180, 180]` degrees for longitude values and `[-90, 90]` degrees for latitude values.

Property values outside the `[minimum, maximum]` range are not allowed, with the exception of `noData` values.

9.3.4.13. Required Properties, No Data Values, and Default Values

When associated property values shall exist for all entities of a class, a property is considered required (`required`).

Individual elements in an array or individual components in a vector or matrix cannot be marked as required; only the property itself can be marked as required.

Properties may optionally specify a No Data value (`noData`, or “sentinel value”) to be used when property values do not exist. A `noData` value may be provided for any type except `BOOLEAN`. For `ENUM` types, a `noData` value should contain the name of the enum value as a string, rather than its integer value. The structure of the `noData` value is explained in the Property Values Structure section.

A `noData` value is especially useful when only some entities in a property table are missing property values (see Binary Table Format). Otherwise if all entities are missing property values the column may be omitted from the table and a `noData` value need not be provided. Entities encoded in the JSON Format may omit the property instead of providing a `noData` value. `noData` values and omitted properties are functionally equivalent.

A default value (`default`) may be provided for missing property values. For `ENUM` types, a `default` value should contain the name of the enum value as a string, rather than its integer value. For all other cases, the structure of the `default` value is explained in the Property Values Structure section.

If a default value is not provided, the behavior when encountering missing property values is implementation-defined.

NOTE: In the example below, a “tree” class is defined with `noData` indicating a specific enum value to be interpreted as missing data.

Properties and types of a tree

PROPERTY	COMPONENTTYPE	REQUIRED	NODATA
height	FLOAT32	Yes	
species	ENUM		"Unknown"
leafColor	STRING	Yes	

9.3.4.14. Property Values Structure

Property values that appear as part of the class definition are the offset, scale, minimum, maximum, default values and no-data values. The structure of these values inside the class definition depends on the type of the property. For SCALAR (non-array) types, they are single values. For all other cases, they are arrays:

- For SCALAR array types with fixed length count, they are arrays with length count.
- For VECN types, they are arrays, with length N.
- For MATN types, they are arrays, with length $N * N$.
- For VECN array types with fixed length count, they are arrays with length count, where each array element is itself an array of length N
- For MATN array types with fixed length count, they are arrays with length count, where each array element is itself an array of length $N * N$.

For noData values and numeric values that are not normalized, the type of the innermost elements of these arrays corresponds to the componentType. For numeric values that are normalized, the innermost elements are floating-point values.

9.4. Storage Formats

9.4.1. Overview

Schemas provide templates for entities, but creating an entity requires specific property values and storage. This section covers two storage formats for entity metadata:

- **Binary Table Format** — property values are stored in parallel 1D arrays, encoded as binary data

- **JSON Format** — property values are stored in key/value dictionaries, encoded as JSON objects

Both formats are suitable for general purpose metadata storage. Binary formats may be preferable for larger quantities of metadata.

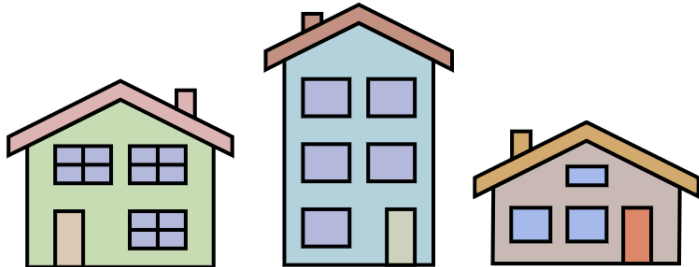
Additional serialization methods may be defined outside of this specification. For example, property values could be stored in texture channels or retrieved from a REST API as XML data.

NOTE: Any specification that references 3D Metadata shall state explicitly which storage formats are supported, or define its own serialization. For example, the [EXT_structural_metadata](#) glTF extension implements the binary table format described below, and defines an additional image-based format for per-textel metadata.

9.4.2. Binary Table Format

9.4.2.1. Overview

The binary table format is similar to a database table where entities are rows and properties are columns. Each column represents one of the properties of the class. Each row represents a single entity conforming to the class.



ID	Name	Stories
0	Green House	2
1	Blue House	3
2	Brown House	2

Figure 2 — Illustration of metadata that can be stored in a table

The rows of a table are addressed by an integer index called an **entity ID**. Entity IDs are always numbered 0, 1, ..., N - 1 where N is the number of rows in the table.

Property values are stored in parallel arrays called **property arrays**, one per column. Each property array stores values for a single property. The *i*-th value of each property array is the value of that property for the entity with an entity ID of *i*.

Binary encoding is efficient for runtime use, and scalable to large quantities of metadata. Because property arrays contain elements of a single type, bitstreams may be tightly packed or may use compression methods appropriate for a particular data type.

Property values are binary-encoded according to their data type, in little-endian format. Values are tightly packed: there is no padding between values.

9.4.2.2. Scalars

A scalar value is encoded based on the `componentType`. Multiple values are packed tightly in the same buffer. The following data types are supported:

Table 8 — Types for scalar metadata values

NAME	DESCRIPTION
INT8	8-bit two's complement signed integer
UINT8	8-bit unsigned integer
INT16	16-bit two's complement signed integer
UINT16	16-bit unsigned integer
INT32	32-bit two's complement signed integer
UINT32	32-bit unsigned integer
INT64	64-bit two's complement signed integer
UINT64	64-bit unsigned integer
FLOAT32	32-bit IEEE floating point number
FLOAT64	64-bit IEEE floating point number

9.4.2.3. Vectors

Vector components are tightly packed and encoded based on the `componentType`.

9.4.2.4. Matrices

Matrix components are tightly packed in column-major order and encoded based on the `componentType`.

9.4.2.5. Booleans

A boolean value is encoded as a single bit, either 0 (`false`) or 1 (`true`). Multiple boolean values are packed tightly in the same buffer. These buffers of tightly-packed bits are sometimes referred to as bitstreams.

For a table with N rows, the buffer that stores these boolean values will consist of $\text{ceil}(N / 8)$ bytes. When N is not divisible by 8, then the unused bits of the last byte of this buffer shall be set to 0.

NOTE: Example accessing a boolean value for entity ID i .

```
byteIndex = floor(i / 8)
bitIndex = i % 8
bitValue = (buffer[byteIndex] >> bitIndex) & 1
value = bitValue == 1
```

9.4.2.6. Strings

A string value is a UTF-8 encoded byte sequence. Multiple strings are packed tightly in the same buffer.

Because string lengths may vary, a **string offset** buffer is used to identify strings in the buffer. If there are N strings in the property array, the string offset buffer has $N + 1$ elements. The first N of these point to the first byte of each string, while the last points to the byte immediately after the last string. The number of bytes in the i -th string is given by $\text{stringOffset}[i + 1] - \text{stringOffset}[i]$. UTF-8 encodes each character as 1-4 bytes, so string offsets do not necessarily represent the number of characters in the string.

The data type used for offsets is defined by a **string offset type**, which may be `UINT8`, `UINT16`, `UINT32`, or `UINT64`.

NOTE: Three UTF-8 strings, binary-encoded in a buffer.

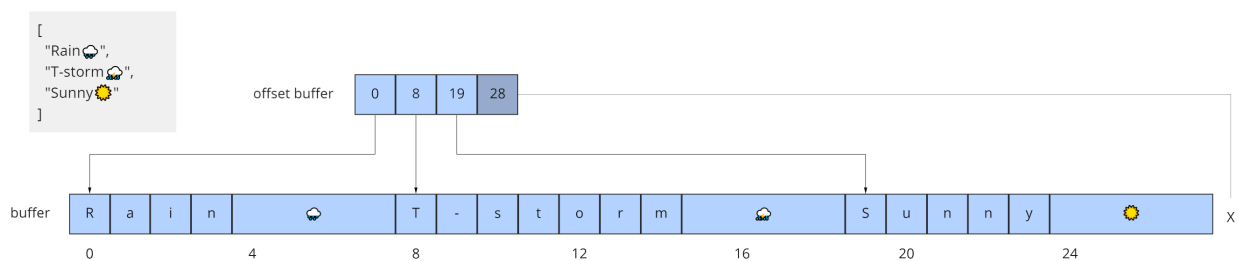


Figure 3 – Data layout for the buffers storing string metadata

9.4.2.7. Enums

Enums are encoded as integer values according to the enum value type (see Enums). Any integer data type supported for Scalars may be used for enum values.

9.4.2.8. Fixed-Length Arrays

A fixed-length array value is encoded as a tightly packed array of count elements, where each element is encoded according to the type.

9.4.2.9. Variable-Length Arrays

Variable-length arrays use an additional **array offset** buffer. The *i*-th value in the array offset buffer is an element index — not a byte offset — identifying the beginning of the *i*-th array. String values within an array may have inconsistent lengths, requiring both array offset and **string offset** buffers (see: Strings).

The data type used for offsets is defined by an **array offset type**, which may be UINT8, UINT16, UINT32, or UINT64.

If there are *N* arrays in the property array, the array offset buffer has *N* + 1 elements. The first *N* of these point to the first element of an array within the property array, or within a string offset buffer for string component types. The last value points to a (non-existent) element immediately following the last array element.

For each case below, the offset of an array element *i* within its binary storage is expressed in terms of entity ID *id* and element index *i*.

Table 9 — Offset types for strings and arrays

TYPE	OFFSET TYPE	OFFSET
STRING	byte offset	<code>stringOffset[arrayOffset[id] + i]</code>
All other types	array index	<code>arrayOffset[id] + i</code>

Each expression in the table above defines an index into the underlying property array. For a property array of SCALAR elements with FLOAT32 component type, index 3 corresponds to byte offset $3 * \text{sizeof}(\text{FLOAT32})$. For a property array of VEC4 elements with FLOAT32 component type, index 3 corresponds to byte offset $3 * 4 * \text{sizeof}(\text{FLOAT32}) = 48$. For an array of BOOLEAN elements, offset 3 would correspond to *bit* offset 3.

NOTE 1: Five variable-length arrays of UINT8 components, binary-encoded in a buffer. The associated property definition would be `type = "SCALAR", componentType = "UINT8", and array = true`.

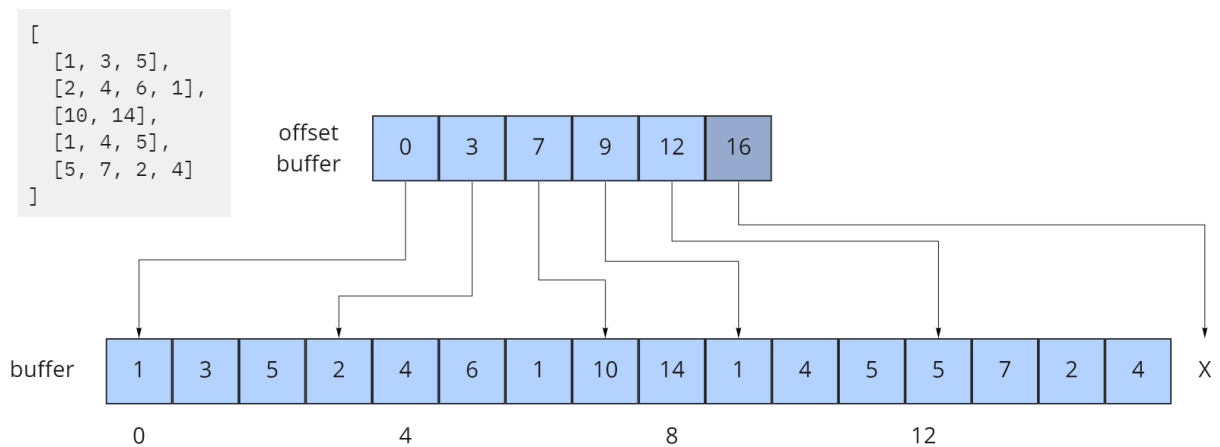


Figure 4 — Data layout for the buffers storing string variable-length arrays

NOTE 2: Two variable-length arrays of strings, binary-encoded in a buffer. The associated property definition would be `type = "STRING"` and `array = true` (variable-length). Observe that the last element of the array offset buffer points to the last element of the string offset buffer. This is because the last valid string offset is the next-to-last element of the string offset buffer.

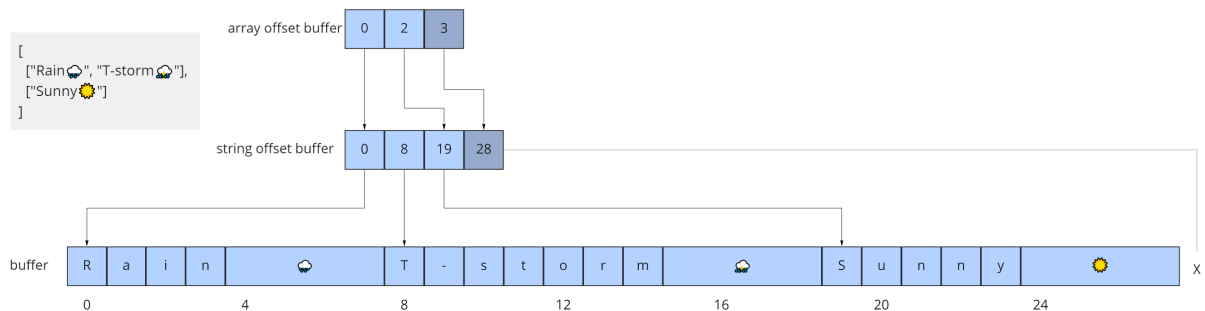


Figure 5 — Data layout for the buffers storing string variable-length arrays of strings

9.4.3. JSON Format

9.4.3.1. Overview

JSON encoding is useful for storing a small number of entities in human readable form.

Each entity is represented as a JSON object with its `class` identified by a string ID. Property values are defined in a key/value properties dictionary, having property IDs as its keys. Property values are encoded as corresponding JSON types: numeric types are represented as number, booleans as boolean, strings as string, enums as string, vectors and matrices as array of number, and arrays as array of the containing type.

NOTE: The following example demonstrates usage for both fixed- and variable-length arrays:

An enum, “basicEnum”, composed of three (name: value) pairs:

Names and values of an example enum

NAME	VALUE
"Enum A"	0
"Enum B"	1
"Enum C"	2

A class, “basicClass”, composed of ten properties. *stringArrayProperty* count is undefined and therefore variable-length.

Properties of an example class

ID	TYPE	COMPONENT	ARRAY	COUNT	ENUMTYPE	REQUIRED
floatProperty	SCALAR	FLOAT64	false			Yes
integerProperty	SCALAR	INT32	false			Yes
vectorProperty	VEC2	FLOAT32	false			Yes
floatArrayProperty	SCALAR	FLOAT32	true	3		Yes
vectorArrayProperty	VEC2	FLOAT32	true	2		Yes
booleanProperty	BOOLEAN		false			Yes
stringProperty	STRING		false			Yes
enumProperty	ENUM		false		basicEnum	Yes
stringArrayProperty	STRING		true			Yes
optionalProperty	STRING		false			

A single entity encoded in JSON. Note that the optional property is omitted in this example.

```
{
  "entity": {
    "class": "basicClass",
    "properties": {
```

```

        "floatProperty": 1.5,
        "integerProperty": -90,
        "vectorProperty": [0.0, 1.0],
        "floatArrayProperty": [1.0, 0.5, -0.5],
        "vectorArrayProperty": [[0.0, 1.0], [1.0, 2.0]],
        "booleanProperty": true,
        "stringProperty": "x123",
        "enumProperty": "Enum B",
        "stringArrayProperty": ["abc", "12345", "おはようございます"]
    }
}

```

9.4.3.2. Scalars

All component types (INT8, UINT8, INT16, UINT16, INT32, UINT32, INT64, UINT64, FLOAT32, and FLOAT64) are encoded as JSON numbers. Floating point values shall be representable as IEEE floating point numbers.

NOTE: For numeric types the size in bits is made explicit. Even though JSON only has a single number type for all integers and floating point numbers, the application that consumes the JSON may make a distinction. For example, C and C++ have several different integer types such as `uint8_t`, `uint32_t`. The application is responsible for interpreting the metadata using the type specified in the property definition.

9.4.3.3. Vectors

Vectors are encoded as a JSON array of numbers.

9.4.3.4. Matrices

Matrices are encoded as a JSON array of numbers in column-major order.

9.4.3.5. Booleans

Booleans are encoded as a JSON boolean, either `true` or `false`.

9.4.3.6. Strings

Strings are encoded as JSON strings.

9.4.3.7. Enums

Enums are encoded as JSON strings using the name of the enum value rather than the integer value. Therefore the enum value type, if specified, is ignored for the JSON encoding.

9.4.3.8. Arrays

Arrays are encoded as JSON arrays, where each element is encoded according to the type. When a count is specified, the length of the JSON array shall match the count. Otherwise, for variable-length arrays, the JSON array may be any length, including zero-length.



10

STYLING

10.1. Overview

3D Tiles styles provide concise declarative styling of tileset features. A style defines expressions to evaluate the display of a feature, for example `color` (RGB and translucency) and `show` properties, often based on the feature's properties.

A style may be applied to a tile that doesn't contain features, in which case the tile is treated as an implicit single feature without properties.

While a style may be created for and reference properties of a tileset, a style is independent of a tileset, such that any style can be applied to any tileset.

Styles are defined with JSON and expressions written in a small subset of JavaScript augmented for styling. Additionally, the styling language provides a set of built-in functions to support common math operations.

The following example assigns a color based on building height.

```
{
  "show" : "${Area} > 0",
  "color" : {
    "conditions" : [
      ["${Height} < 60", "color('#13293D')"],
      ["${Height} < 120", "color('#1B98E0')"],
      ["true", "color('#E8F1F2', 0.5)"]
    ]
  }
}
```

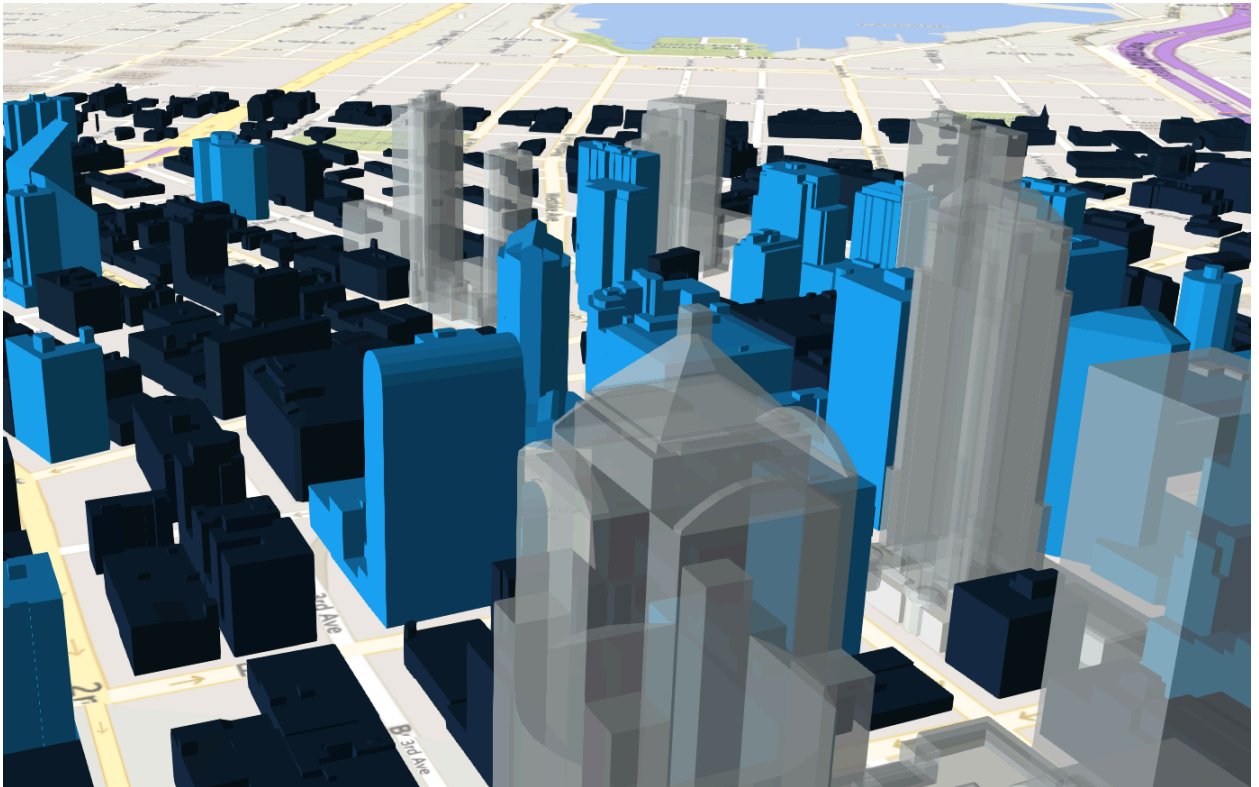


Figure 6 — Rendering of buildings with different colors, based on the height of the buildings

10.2. Concepts

10.2.1. Styling features

Visual properties available for styling features are the `show` property, the assigned expression of which will evaluate to a boolean that determines if the feature is visible, and the `color` property, the assigned expression of which will evaluate to a `Color` object (RGB and translucency) which determines the displayed color of a feature.

The following style assigns the default `show` and `color` properties to each feature:

```
{
  "show" : "true",
  "color" : "color('#ffffff')"
}
```

Instead of showing all features, `show` can be an expression dependent on a feature's properties, for example, the following expression will show only features in the 19341 zip code:

```
{
  "show" : "${ZipCode} === '19341'"
}
```

```
}
```

show can also be used for more complex queries; for example, here a compound condition and regular expression are used to show only features whose county starts with 'Chest' and whose year built is greater than or equal to 1970:

```
{
  "show" : "(RegExp('^Chest').test(${County})) && (${YearBuilt} >= 1970)"
}
```

Colors can also be defined by expressions dependent on a feature's properties. For example, the following expression colors features with a temperature above 90 as red and the others as white:

```
{
  "color" : "(${Temperature} > 90) ? color('red') : color('white')"
}
```

The color's alpha component defines the feature's opacity. For example, the following sets the feature's RGB color components from the feature's properties and makes features with volume greater than 100 transparent:

```
{
  "color" : "rgba(${red}, ${green}, ${blue}, (${volume} > 100 ? 0.5 : 1.0))"
}
```

10.2.2. Conditions

In addition to a string containing an expression, color and show can be an array defining a series of conditions (similar to if...else statements). Conditions can, for example, be used to make color maps and color ramps with any type of inclusive/exclusive intervals.

For example, the following expression maps an ID property to colors. Conditions are evaluated in order, so if \${id} is not '1' or '2', the "true" condition returns white. If no conditions are met, the color of the feature will be undefined:

```
{
  "color" : {
    "conditions" : [
      ["${id} === '1'", "color('#FF0000')"],
      ["${id} === '2'", "color('#00FF00')"],
      ["true", "color('#FFFFFF')"]
    ]
  }
}
```

The next example shows how to use conditions to create a color ramp using intervals with an inclusive lower bound and exclusive upper bound:

```
"color" : {
  "conditions" : [
    ["(${Height} >= 1.0) && (${Height} < 10.0)", "color('#FF00FF')"],
    ["(${Height} >= 10.0) && (${Height} < 30.0)", "color('#FF0000')"],
    ["(${Height} >= 30.0) && (${Height} < 50.0)", "color('#FFFF00')"],
    ["(${Height} >= 50.0) && (${Height} < 70.0)", "color('#00FF00')"],
    ["(${Height} >= 70.0) && (${Height} < 100.0)", "color('#00FFFF')"],
    ["(${Height} >= 100.0)", "color('#0000FF')"]
  ]
}
```

```
}
```

Since conditions are evaluated in order, the above can be written more concisely as the following:

```
"color" : {
  "conditions" : [
    ["({Height} >= 100.0)", "color('#0000FF')"],
    ["({Height} >= 70.0)", "color('#00FFFF')"],
    ["({Height} >= 50.0)", "color('#00FF00')"],
    ["({Height} >= 30.0)", "color('#FFFF00')"],
    ["({Height} >= 10.0)", "color('#FF0000')"],
    ["({Height} >= 1.0)", "color('#FF00FF')"]
  ]
}
```

10.2.3. Defining variables

Commonly used expressions may be stored in a defines object with a variable name as a key. If a variable references the name of a defined expression, it is replaced with the result of the referenced evaluated expression:

```
{
  "defines" : {
    "NewHeight" : "clamp(({Height} - 0.5) / 2.0, 1.0, 255.0)",
    "HeightColor" : "rgb({Height}, {Height}, {Height})"
  },
  "color" : {
    "conditions" : [
      ["({NewHeight} >= 100.0)", "color('#0000FF') * ${HeightColor}"],
      ["({NewHeight} >= 50.0)", "color('#00FF00') * ${HeightColor}"],
      ["({NewHeight} >= 1.0)", "color('#FF0000') * ${HeightColor}"]
    ]
  },
  "show" : "${NewHeight} < 200.0"
}
```

A define expression may not reference other defines; however, it may reference feature properties with the same name. In the style below a feature of height 150 gets the color red:

```
{
  "defines" : {
    "Height" : "${Height}/2.0",
  },
  "color" : {
    "conditions" : [
      ["({Height} >= 100.0)", "color('#0000FF')"],
      ["({Height} >= 1.0)", "color('#FF0000')"]
    ]
  }
}
```

10.2.4. Meta property

Non-visual properties of a feature can be defined using the meta property. For example, the following sets a description meta property to a string containing the feature name:

```
{
  "meta" : {
    "description" : "'Hello, ${featureName}.'"
  }
}
```

A meta property expression can evaluate to any type. For example:

```
{
  "meta" : {
    "featureColor" : "rgb(${red}, ${green}, ${blue})",
    "featureVolume" : "${height} * ${width} * ${depth}"
  }
}
```

10.3. Expressions

The language for expressions is a small subset of JavaScript ([EMCAScript 5](#)), plus native vector and regular expression types and access to tileset feature properties in the form of readonly variables.

NOTE: CesiumJS uses the [jsep](#) JavaScript expression parser library to parse style expressions into an [abstract syntax tree \(AST\)](#).

10.3.1. Semantics

Dot notation is used to access properties by name, e.g., `building.name`.

Bracket notation (`[]`) is also used to access properties, e.g., `building['name']`, or arrays, e.g., `temperatures[1]`.

Functions are called with parenthesis (`()`) and comma-separated arguments, e.g., `isNaN(0.0)`, `color('cyan', 0.5)`.

10.3.2. Operators

The following operators are supported with the same semantics and precedence as JavaScript.

- Unary: `+`, `-`, `!`
 - Not supported: `~`
- Binary: `|`, `&`, `==`, `!=`, `<`, `>`, `<=`, `>=`, `+`, `-`, `*`, `/`, `%`, `=~`, `!~`
 - Not supported: `|`, `^`, `&`, `<<`, `>>`, and `>>>`

- Ternary: `? :`

(`and`) are also supported for grouping expressions for clarity and precedence.

Logical `||` and `&&` implement short-circuiting; `true ||` expression does not evaluate the right expression, and `false &&` expression does not evaluate the right expression.

Similarly, `true ? leftExpression : rightExpression` only executes the left expression, and `false ? leftExpression : rightExpression` only executes the right expression.

10.3.3. Types

The following types are supported:

- `Boolean`
- `Null`
- `Undefined`
- `Number`
- `String`
- `Array`
- `vec2`
- `vec3`
- `vec4`
- `RegExp`

All of the types except `vec2`, `vec3`, `vec4`, and `RegExp` have the same syntax and runtime behavior as JavaScript. `vec2`, `vec3`, and `vec4` are derived from GLSL vectors and behave similarly to JavaScript `Object` (see the [Vector](#) section). Colors derive from [CSS3 Colors](#) and are implemented as `vec4`. `RegExp` is derived from JavaScript and described in the [RegExp](#) section.

Example expressions for different types include the following:

- `true, false`
- `null`
- `undefined`
- `1.0, NaN, Infinity`
- `'Cesium', "Cesium"`

- `[0, 1, 2]`
- `vec2(1.0, 2.0)`
- `vec3(1.0, 2.0, 3.0)`
- `vec4(1.0, 2.0, 3.0, 4.0)`
- `color('#00FFFF')`
- `regExp('^Chest')`

10.3.3.1. Number

As in JavaScript, numbers can be NaN or Infinity. The following test functions are supported:

- `isNaN(testValue : Number) : Boolean`
- `isFinite(testValue : Number) : Boolean`

10.3.3.2. String

Strings are encoded in UTF-8.

10.3.3.3. Vector

The styling language includes 2, 3, and 4 component floating-point vector types: `vec2`, `vec3`, and `vec4`. Vector constructors share the same rules as GLSL:

10.3.3.3.1. `vec2`

- `vec2(xy : Number)` — initialize each component with the number
- `vec2(x : Number, y : Number)` — initialize with two numbers
- `vec2(xy : vec2)` — initialize with another `vec2`
- `vec2(xyz : vec3)` — drops the third component of a `vec3`
- `vec2(xyzw : vec4)` — drops the third and fourth component of a `vec4`

10.3.3.3.2. `vec3`

- `vec3(xyz : Number)` — initialize each component with the number

- `vec3(x : Number, y : Number, z : Number)` — initialize with three numbers
- `vec3(xyz : vec3)` — initialize with another `vec3`
- `vec3(xyzw : vec4)` — drops the fourth component of a `vec4`
- `vec3(xy : vec2, z : Number)` — initialize with a `vec2` and number
- `vec3(x : Number, yz : vec2)` — initialize with a `vec2` and number

10.3.3.3.3. `vec4`

- `vec4(xyzw : Number)` — initialize each component with the number
- `vec4(x : Number, y : Number, z : Number, w : Number)` — initialize with four numbers
- `vec4(xyzw : vec4)` — initialize with another `vec4`
- `vec4(xy : vec2, z : Number, w : Number)` — initialize with a `vec2` and two numbers
- `vec4(x : Number, yz : vec2, w : Number)` — initialize with a `vec2` and two numbers
- `vec4(x : Number, y : Number, zw : vec2)` — initialize with a `vec2` and two numbers
- `vec4(xyz : vec3, w : Number)` — initialize with a `vec3` and number
- `vec4(x : Number, yzw : vec3)` — initialize with a `vec3` and number

10.3.3.3.4. Vector usage

`vec2` components may be accessed with

- `.x, .y`
- `.r, .g`
- `[0], [1]`

`vec3` components may be accessed with

- `.x, .y, .z`
- `.r, .g, .b`
- `[0], [1], [2]`

`vec4` components may be accessed with

- `.x, .y, .z, .w`
- `.r, .g, .b, .a`
- `[0], [1], [2], [3]`

Unlike GLSL, the styling language does not support swizzling. For example, `vec3(1.0).xy` is not supported.

Vectors support the following unary operators: `-`, `+`.

Vectors support the following binary operators by performing component-wise operations: `=`, `!=`, `+`, `-`, `*`, `/`, and `%`. For example `vec4(1.0) === vec4(1.0)` is true since the x, y, z, and w components are equal. Operators are essentially overloaded for `vec2`, `vec3`, and `vec4`.

`vec2`, `vec3`, and `vec4` have a `toString` function for explicit (and implicit) conversion to strings in the format `'(x, y)'`, `'(x, y, z)'`, and `'(x, y, z, w)'`.

- `toString() : String`

`vec2`, `vec3`, and `vec4` do not expose any other functions or a prototype object.

10.3.3.4. Color

Colors are implemented as `vec4` and are created with one of the following functions:

- `color()`
- `color(keyword : String, [alpha : Number])`
- `color(6-digit-hex : String, [alpha : Number])`
- `color(3-digit-hex : String, [alpha : Number])`
- `rgb(red : Number, green : Number, blue : Number)`
- `rgba(red : Number, green : Number, blue : Number, alpha : Number)`
- `hsl(hue : Number, saturation : Number, lightness : Number)`
- `hsla(hue : Number, saturation : Number, lightness : Number, alpha : Number)`

Calling `color()` with no arguments is the same as calling `color('#FFFFFF')`.

Colors defined by a case-insensitive keyword (e.g., `'cyan'`) or hex rgb are passed as strings to the `color` function. For example:

- `color('cyan')`
- `color('#00FFFF')`

- `color('#0FF')`

These color functions have an optional second argument that is an alpha component to define opacity, where 0.0 is fully transparent and 1.0 is fully opaque. For example:

- `color('cyan', 0.5)`

Colors defined with decimal RGB or HSL are created with `rgb` and `hsl` functions, respectively, just as in CSS (but with percentage ranges from 0.0 to 1.0 for 0% to 100%, respectively). For example:

- `rgb(100, 255, 190)`
- `hsl(1.0, 0.6, 0.7)`

The range for `rgb` components is 0 to 255, inclusive. For `hsl`, the range for hue, saturation, and lightness is 0.0 to 1.0, inclusive.

Colors defined with `rgba` or `hsla` have a fourth argument that is an alpha component to define opacity, where 0.0 is fully transparent and 1.0 is fully opaque. For example:

- `rgba(100, 255, 190, 0.25)`
- `hsla(1.0, 0.6, 0.7, 0.75)`

Colors are equivalent to the `vec4` type and share the same functions, operators, and component accessors. Color components are stored in the range 0.0 to 1.0.

For example:

- `color('red').x`, `color('red').r`, and `color('red')[0]` all evaluate to 1.0.
- `color('red').toString()` evaluates to `(1.0, 0.0, 0.0, 1.0)`
- `color('red') * vec4(0.5)` is equivalent to `vec4(0.5, 0.0, 0.0, 0.5)`

10.3.3.5. RegExp

Regular expressions are created with the following functions, which behave like the JavaScript `RegExp` constructor:

- `regex()`
- `regex(pattern : String, [flags : String])`

Calling `regex()` with no arguments is the same as calling `regex('(?:)')`.

If specified, flags can have any combination of the following values:

- `g` — global match
- `i` — ignore case
- `m` — multiline
- `u` — unicode
- `y` — sticky

Regular expressions support these functions:

- `test(string : String) : Boolean` — Tests the specified string for a match.
- `exec(string : String) : String` — Executes a search for a match in the specified string. If the search succeeds, it returns the first instance of a captured `String`. If the search fails, it returns `null`.

For example:

```
{
  "Name" : "Building 1"
}

RegExp('a').test('abc') === true
RegExp('a(.)', 'i').exec('Abc') === 'b'
RegExp('Building\s(\d)').exec(${Name}) === '1'
```

Regular expressions have a `toString` function for explicit (and implicit) conversion to strings in the format `'pattern'`:

- `toString() : String`

Regular expressions do not expose any other functions or a prototype object.

The operators `=~` and `!~` are overloaded for regular expressions. The `=~` operator matches the behavior of the `test` function, and tests the specified string for a match. It returns `true` if one is found, and `false` if not found. The `!~` operator is the inverse of the `=~` operator. It returns `true` if no matches are found, and `false` if a match is found. Both operators are commutative.

For example, the following expressions all evaluate to `true`:

```
RegExp('a') =~ 'abc'
'abc' =~ RegExp('a')

RegExp('a') !~ 'bcd'
'bcd' !~ RegExp('a')
```

10.3.4. Operator rules

- Unary operators + and - operate only on number and vector expressions.
- Unary operator ! operates only on boolean expressions.
- Binary operators <, <=, >, and >= operate only on number expressions.
- Binary operators || and && operate only on boolean expressions.
- Binary operator + operates on the following expressions:
 - Number expressions
 - Vector expressions of the same type
 - If at least one expressions is a string, the other expression is converted to a string following String Conversions, and the operation returns a concatenated string, e.g. "name" + 10 evaluates to "name10"
- Binary operator - operates on the following expressions:
 - Number expressions
 - Vector expressions of the same type
- Binary operator * operates on the following expressions:
 - Number expressions
 - Vector expressions of the same type
 - Mix of number expression and vector expression, e.g. 3 * vec3(1.0) and vec2(1.0) * 3
- Binary operator / operates on the following expressions:
 - Number expressions
 - Vector expressions of the same type
 - Vector expression followed by number expression, e.g. vec3(1.0) / 3
- Binary operator % operates on the following expressions:
 - Number expressions
 - Vector expressions of the same type

- Binary equality operators `===` and `!==` operate on any expressions. The operation returns false if the expression types do not match.
- Binary `RegExp` operators `=~` and `!~` require one argument to be a string expression and the other to be a `RegExp` expression.
- Ternary operator `?` : conditional argument shall be a boolean expression.

10.3.5. Type conversions

Explicit conversions between primitive types are handled with `Boolean`, `Number`, and `String` functions.

- `Boolean(value : Any) : Boolean`
- `Number(value : Any) : Number`
- `String(value : Any) : String`

For example:

```
Boolean(1) === true
Number('1') === 1
String(1) === '1'
```

`Boolean` and `Number` follow JavaScript conventions. `String` follows String Conversions.

These are essentially casts, not constructor functions.

The styling language does not allow for implicit type conversions, unless stated above. Expressions like `vec3(1.0) === vec4(1.0)` and `"5" < 6` are not valid.

10.3.6. String conversions

`vec2`, `vec3`, `vec4`, and `RegExp` expressions are converted to strings using their `toString` methods. All other types follow JavaScript conventions.

- `true` — `"true"`
- `false` — `"false"`
- `null` — `"null"`
- `undefined` — `"undefined"`
- `5.0` — `"5"`
- `NaN` — `"NaN"`
- `Infinity` — `"Infinity"`

- "name" — "name"
- [0, 1, 2] — "[0, 1, 2]"
- vec2(1, 2) — "(1, 2)"
- vec3(1, 2, 3) — "(1, 2, 3)"
- vec4(1, 2, 3, 4) — "(1, 2, 3, 4)"
- RegExp('a') — "/a/"

10.3.7. Constants

The following constants are supported by the styling language:

- Math.PI
- Math.E

10.3.7.1. PI

The mathematical constant PI, which represents a circle's circumference divided by its diameter, approximately 3.14159.

```
{
  "show" : "cos(${Angle} + Math.PI) < 0"
}
```

10.3.7.2. E

Euler's constant and the base of the natural logarithm, approximately 2.71828.

```
{
  "color" : "color() * pow(Math.E / 2.0, ${Temperature})"
}
```

10.3.8. Variables

Variables are used to retrieve the property values of individual features in a tileset. These properties are sourced from the [EXT_structural_metadata](#) glTF extension, which provides property textures and property tables for feature attributes. Variables are identified using the ES 6 (ECMAScript 2015) template literal syntax, i.e., `${feature.identifier}` or `${feature['identifier']}`, where the identifier is the case-sensitive property name. Variable names are encoded in UTF-8. `feature` is implicit and can be omitted in most cases. If the identifier contains non-alphanumeric characters, such as `:`, `-`, `#`, or spaces, the `${feature['identifier']}` form should be used.

Variables can be used anywhere a valid expression is accepted, except inside other variable identifiers. For example, the following is not allowed:

```
`${foo[${bar}]}`  
----
```

If a feature does not have a property with the specified name, the variable evaluates to `undefined`. Note that the property may also be `null` if `null` was explicitly stored for a property.

Variables may be any of the supported native JavaScript types:

- Boolean
- Null
- Undefined
- Number
- String
- Array

For example:

```
{  
  "enabled" : true,  
  "description" : null,  
  "order" : 1,  
  "name" : "Feature name"  
}  
  
${enabled} === true  
${description} === null  
${order} === 1  
${name} === 'Feature name'
```

Additionally, variables representing vector properties are treated as vector types:

Variables can be used to construct colors or vectors. For example:

```
rgba(${red}, ${green}, ${blue}, ${alpha})  
vec4(${temperature})
```

Dot or bracket notation is used to access feature subproperties. For example:

```
{  
  "address" : {  
    "street" : "Example street",  
    "city" : "Example city"  
  }  
}  
  
${address.street} === `Example street`  
${address['street']} === `Example street`  
  
${address.city} === `Example city`  
${address['city']} === `Example city`
```

Bracket notation supports only string literals.

Top-level properties can be accessed with bracket notation by explicitly using the feature keyword. For example:

```
{
  "address.street" : "Maple Street",
  "address" : {
    "street" : "Oak Street"
  }
}

${address.street} === `Oak Street`
${feature.address.street} === `Oak Street`
${feature['address'].street} === `Oak Street`
${feature['address.street']} === `Maple Street`
```

To access a feature named feature, use the variable `${feature}`. This is equivalent to accessing `${feature.feature}`

```
{
  "feature" : "building"
}

${feature} === `building`
${feature.feature} === `building`
```

Variables can also be substituted inside strings defined with backticks, for example:

```
{
  "order" : 1,
  "name" : "Feature name"
}

`Name is ${name}, order is ${order}`
```

Bracket notation is used to access feature subproperties or arrays. For example:

```
{
  "temperatures" : {
    "scale" : "fahrenheit",
    "values" : [70, 80, 90]
  }
}

${temperatures['scale']} === 'fahrenheit'
${temperatures.values[0]} === 70
${temperatures['values'][0]} === 70 // Same as (temperatures[values])[0] and
temperatures.values[0]
```

10.3.9. Built-in functions

The following built-in functions are supported by the styling language:

- abs
- sqrt
- cos
- sin

- `tan`
- `acos`
- `asin`
- `atan`
- `atan2`
- `radians`
- `degrees`
- `sign`
- `floor`
- `ceil`
- `round`
- `exp`
- `log`
- `exp2`
- `log2`
- `fract`
- `pow`
- `min`
- `max`
- `clamp`
- `mix`
- `length`
- `distance`
- `normalize`
- `dot`
- `cross`

Many of the built-in functions take either scalars or vectors as arguments. For vector arguments the function is applied component-wise and the resulting vector is returned.

10.3.9.1. abs

```
abs(x : Number) : Number
abs(x : vec2) : vec2
abs(x : vec3) : vec3
abs(x : vec4) : vec4
```

Returns the absolute value of x.

```
{
  "show" : "abs(${temperature}) > 20.0"
}
```

10.3.9.2. sqrt

```
sqrt(x : Number) : Number
sqrt(x : vec2) : vec2
sqrt(x : vec3) : vec3
sqrt(x : vec4) : vec4
```

Returns the square root of x when x >= 0. Returns NaN when x < 0.

```
{
  "color" : {
    "conditions" : [
      ["${temperature} >= 0.5", "color('#00FFFF)"],
      ["${temperature} >= 0.0", "color('#FF00FF)"]
    ]
  }
}
```

10.3.9.3. cos

```
cos(angle : Number) : Number
cos(angle : vec2) : vec2
cos(angle : vec3) : vec3
cos(angle : vec4) : vec4
```

Returns the cosine of angle in radians.

```
{
  "show" : "cos(${Angle}) > 0.0"
}
```

10.3.9.4. sin

```
sin(angle : Number) : Number
sin(angle : vec2) : vec2
sin(angle : vec3) : vec3
sin(angle : vec4) : vec4
```

Returns the sine of angle in radians.

```
{
```

```

    "show" : "sin(${Angle}) > 0.0"
}

```

10.3.9.5. tan

```

tan(angle : Number) : Number
tan(angle : vec2) : vec2
tan(angle : vec3) : vec3
tan(angle : vec4) : vec4

```

Returns the tangent of angle in radians.

```

{
    "show" : "tan(${Angle}) > 0.0"
}

```

10.3.9.6. acos

```

acos(angle : Number) : Number
acos(angle : vec2) : vec2
acos(angle : vec3) : vec3
acos(angle : vec4) : vec4

```

Returns the arccosine of angle in radians.

```

{
    "show" : "acos(${Angle}) > 0.0"
}

```

10.3.9.7. asin

```

asin(angle : Number) : Number
asin(angle : vec2) : vec2
asin(angle : vec3) : vec3
asin(angle : vec4) : vec4

```

Returns the arcsine of angle in radians.

```

{
    "show" : "asin(${Angle}) > 0.0"
}

```

10.3.9.8. atan

```

atan(angle : Number) : Number
atan(angle : vec2) : vec2
atan(angle : vec3) : vec3
atan(angle : vec4) : vec4

```

Returns the arctangent of angle in radians.

```

{
    "show" : "atan(${Angle}) > 0.0"
}

```

10.3.9.9. atan2

```
atan2(y : Number, x : Number) : Number
atan2(y : vec2, x : vec2) : vec2
atan2(y : vec3, x : vec3) : vec3
atan2(y : vec4, x : vec4) : vec4
```

Returns the arctangent of the quotient of y and x.

```
{
  "show" : "atan2(${GridY}, ${GridX}) > 0.0"
}
```

10.3.9.10. radians

```
radians(angle : Number) : Number
radians(angle : vec2) : vec2
radians(angle : vec3) : vec3
radians(angle : vec4) : vec4
```

Converts angle from degrees to radians.

```
{
  "show" : "radians(${Angle}) > 0.5"
}
```

10.3.9.11. degrees

```
degrees(angle : Number) : Number
degrees(angle : vec2) : vec2
degrees(angle : vec3) : vec3
degrees(angle : vec4) : vec4
```

Converts angle from radians to degrees.

```
{
  "show" : "degrees(${Angle}) > 45.0"
}
```

10.3.9.12. sign

```
sign(x : Number) : Number
sign(x : vec2) : vec2
sign(x : vec3) : vec3
sign(x : vec4) : vec4
```

Returns 1.0 when x is positive, 0.0 when x is zero, and -1.0 when x is negative.

```
{
  "show" : "sign(${Temperature}) * sign(${Velocity}) === 1.0"
}
```

10.3.9.13. floor

```
floor(x : Number) : Number
floor(x : vec2) : vec2
floor(x : vec3) : vec3
floor(x : vec4) : vec4
```

Returns the nearest integer less than or equal to x.

```
{
  "show" : "floor(${Position}) === 0"
}
```

10.3.9.14. ceil

```
ceil(x : Number) : Number
ceil(x : vec2) : vec2
ceil(x : vec3) : vec3
ceil(x : vec4) : vec4
```

Returns the nearest integer greater than or equal to x.

```
{
  "show" : "ceil(${Position}) === 1"
}
```

10.3.9.15. round

```
round(x : Number) : Number
round(x : vec2) : vec2
round(x : vec3) : vec3
round(x : vec4) : vec4
```

Returns the nearest integer to x. A number with a fraction of 0.5 will round in an implementation-defined direction.

```
{
  "show" : "round(${Position}) === 1"
}
```

10.3.9.16. exp

```
exp(x : Number) : Number
exp(x : vec2) : vec2
exp(x : vec3) : vec3
exp(x : vec4) : vec4
```

Returns e to the power of x, where e is Euler's constant, approximately 2.71828.

```
{
  "show" : "exp(${Density}) > 1.0"
}
```

10.3.9.17. log

```
log(x : Number) : Number  
log(x : vec2) : vec2  
log(x : vec3) : vec3  
log(x : vec4) : vec4
```

Returns the natural logarithm (base e) of x.

```
{  
  "show" : "log(${Density}) > 1.0"  
}
```

10.3.9.18. exp2

```
exp2(x : Number) : Number  
exp2(x : vec2) : vec2  
exp2(x : vec3) : vec3  
exp2(x : vec4) : vec4
```

Returns 2 to the power of x.

```
{  
  "show" : "exp2(${Density}) > 1.0"  
}
```

10.3.9.19. log2

```
log2(x : Number) : Number  
log2(x : vec2) : vec2  
log2(x : vec3) : vec3  
log2(x : vec4) : vec4
```

Returns the base 2 logarithm of x.

```
{  
  "show" : "log2(${Density}) > 1.0"  
}
```

10.3.9.20. fract

```
fract(x : Number) : Number  
fract(x : vec2) : vec2  
fract(x : vec3) : vec3  
fract(x : vec4) : vec4
```

Returns the fractional part of x. Equivalent to $x - \text{floor}(x)$.

```
{  
  "color" : "color() * fract(${Density})"  
}
```

10.3.9.21. pow

```
pow(base : Number, exponent : Number) : Number
pow(base : vec2, exponent : vec2) : vec2
pow(base : vec3, exponent : vec3) : vec3
pow(base : vec4, exponent : vec4) : vec4
```

Returns base raised to the power of exponent.

```
{
  "show" : "pow(${Density}, ${Temperature}) > 1.0"
}
```

10.3.9.22. min

```
min(x : Number, y : Number) : Number
min(x : vec2, y : vec2) : vec2
min(x : vec3, y : vec3) : vec3
min(x : vec4, y : vec4) : vec4

min(x : Number, y : Number) : Number
min(x : vec2, y : Number) : vec2
min(x : vec3, y : Number) : vec3
min(x : vec4, y : Number) : vec4
```

Returns the smaller of x and y.

```
{
  "show" : "min(${Width}, ${Height}) > 10.0"
}
```

10.3.9.23. max

```
max(x : Number, y : Number) : Number
max(x : vec2, y : vec2) : vec2
max(x : vec3, y : vec3) : vec3
max(x : vec4, y : vec4) : vec4

max(x : Number, y : Number) : Number
max(x : vec2, y : Number) : vec2
max(x : vec3, y : Number) : vec3
max(x : vec4, y : Number) : vec4
```

Returns the larger of x and y.

```
{
  "show" : "max(${Width}, ${Height}) > 10.0"
}
```

10.3.9.24. clamp

```
clamp(x : Number, min : Number, max : Number) : Number
clamp(x : vec2, min : vec2, max : vec2) : vec2
clamp(x : vec3, min : vec3, max : vec3) : vec3
```

```

clamp(x : vec4, min : vec4, max : vec4) : vec4
clamp(x : Number, min : Number, max : Number) : Number
clamp(x : vec2, min : Number, max : Number) : vec2
clamp(x : vec3, min : Number, max : Number) : vec3
clamp(x : vec4, min : Number, max : Number) : vec4

```

Constrains x to lie between min and max.

```

{
  "color" : "color() * clamp(${temperature}, 0.1, 0.2)"
}

```

10.3.9.25. mix

```

mix(x : Number, y : Number, a : Number) : Number
mix(x : vec2, y : vec2, a : vec2) : vec2
mix(x : vec3, y : vec3, a : vec3) : vec3
mix(x : vec4, y : vec4, a : vec4) : vec4

mix(x : Number, y : Number, a : Number) : Number
mix(x : vec2, y : vec2, a : Number) : vec2
mix(x : vec3, y : vec3, a : Number) : vec3
mix(x : vec4, y : vec4, a : Number) : vec4

```

Computes the linear interpolation of x and y.

```

{
  "show" : "mix(20.0, ${Angle}, 0.5) > 25.0"
}

```

10.3.9.26. length

```

length(x : Number) : Number
length(x : vec2) : vec2
length(x : vec3) : vec3
length(x : vec4) : vec4

```

Computes the length of vector x, i.e., the square root of the sum of the squared components. If x is a number, length returns x.

```

{
  "show" : "length(${Dimensions}) > 10.0"
}

```

10.3.9.27. distance

```

distance(x : Number, y : Number) : Number
distance(x : vec2, y : vec2) : vec2
distance(x : vec3, y : vec3) : vec3
distance(x : vec4, y : vec4) : vec4

```

Computes the distance between two points x and y, i.e., length(x - y).

```

{
  "show" : "distance(${BottomRight}, ${UpperLeft}) > 50.0"
}

```

10.3.9.28. normalize

```
normalize(x : Number) : Number
normalize(x : vec2) : vec2
normalize(x : vec3) : vec3
normalize(x : vec4) : vec4
```

Returns a vector with length 1.0 that is parallel to x. When x is a number, normalize returns 1.0.

```
{
  "show" : "normalize(${RightVector}, ${UpVector}) > 0.5"
}
```

10.3.9.29. dot

```
dot(x : Number, y : Number) : Number
dot(x : vec2, y : vec2) : vec2
dot(x : vec3, y : vec3) : vec3
dot(x : vec4, y : vec4) : vec4
```

Computes the dot product of x and y.

```
{
  "show" : "dot(${RightVector}, ${UpVector}) > 0.5"
}
```

10.3.9.30. cross

```
cross(x : vec3, y : vec3) : vec3
```

Computes the cross product of x and y. This function only accepts vec3 arguments.

```
{
  "color" : "vec4(cross(${RightVector}, ${UpVector}), 1.0)"
}
```

10.3.10. Notes

Comments are not supported.

10.4. File extension and media type

Tileset styles use the .json extension and the application/json media type.



ANNEX A (NORMATIVE) ANNEX A (NORMATIVE) 3DTILES_TILESET



ANNEX A (NORMATIVE) ANNEX A (NORMATIVE) 3DTILES_TILESET

A.1. Contributors

- Sean Lilley, Cesium
- Adam Morris, Cesium
- Tamrat Belayneh, ESRI

A.2. Status

Draft

A.3. Dependencies

Written against the glTF 2.1 spec.

A.4. Required

This extension is required, meaning it **MUST** be placed in both `extensionsRequired` and `extensionsUsed`.

A.5. Contents

- [Overview](#)
- [File Extensions](#)
- [Constraints](#)
- [Concepts](#)
 - [3D Tiles](#)
 - [Tileset](#)
 - [Tile](#)
 - [Content](#)
 - [External Tilesets](#)
 - [Geometric Error](#)
 - [Refinement](#)
 - [Bounding Volumes](#)
 - [Transforms](#)
 - [Spatial Coherence](#)
 - [Spatial Data Structures](#)
 - [Coordinate Reference System](#)
 - [Implicit Tiling](#)
- [Supporting Extensions](#)
 - [Metadata](#)
 - [Layers](#)
 - [Horizon Occlusion Point](#)
 - [Visibility Bounding Volumes](#)
 - [Declarative Styling](#)
- [Appendix A: Spatial Data Structures](#)

A.6. Overview

This extension specifies a subset of glTF 2.1 for representing a tileset in 3D Tiles. It extends the glTF node hierarchy to support Hierarchical Level of Detail (HLOD) for streaming massive 3D scenes. Additionally, it depends on core glTF 2.1 features like external assets and bounding volumes.

A.7. File Extensions

Assets that use the 3DTILES_tileset extension **SHOULD** use the .tileset.gltf or .tileset.glb file extensions. This convention helps differentiate tileset files from content files.

The entry tileset **SHOULD** be named root.tileset.gltf or root.tileset.glb to differentiate the entry tileset from [External Tilesets](#).

A.8. Constraints

The following constraints apply when using the 3DTILES_tileset extension:

- The document **MUST** have exactly one scene with exactly one node, the single root node.
- The 3DTILES_tileset extension **MUST** be defined on all nodes.

A.9. Concepts

A.9.1. 3D Tiles

In 3D Tiles, a *tileset* is a set of *tiles* organized in a spatial data structure, the *tree*. Each tile may reference renderable *content*.

The content references a set of *features*, such as 3D models representing buildings or trees, or points in a point cloud. Each feature has position and appearance properties and additional

application-specific properties. A client may choose to select features at runtime and retrieve their properties for visualization or analysis.

Tiles are organized in a tree which incorporates the concept of Hierarchical Level of Detail (HLOD) for optimal rendering of spatial data. Each tile has a *bounding volume*, an object defining a spatial extent completely enclosing its content. The tree has spatial coherence; the content for child tiles are completely inside the parent's bounding volume.

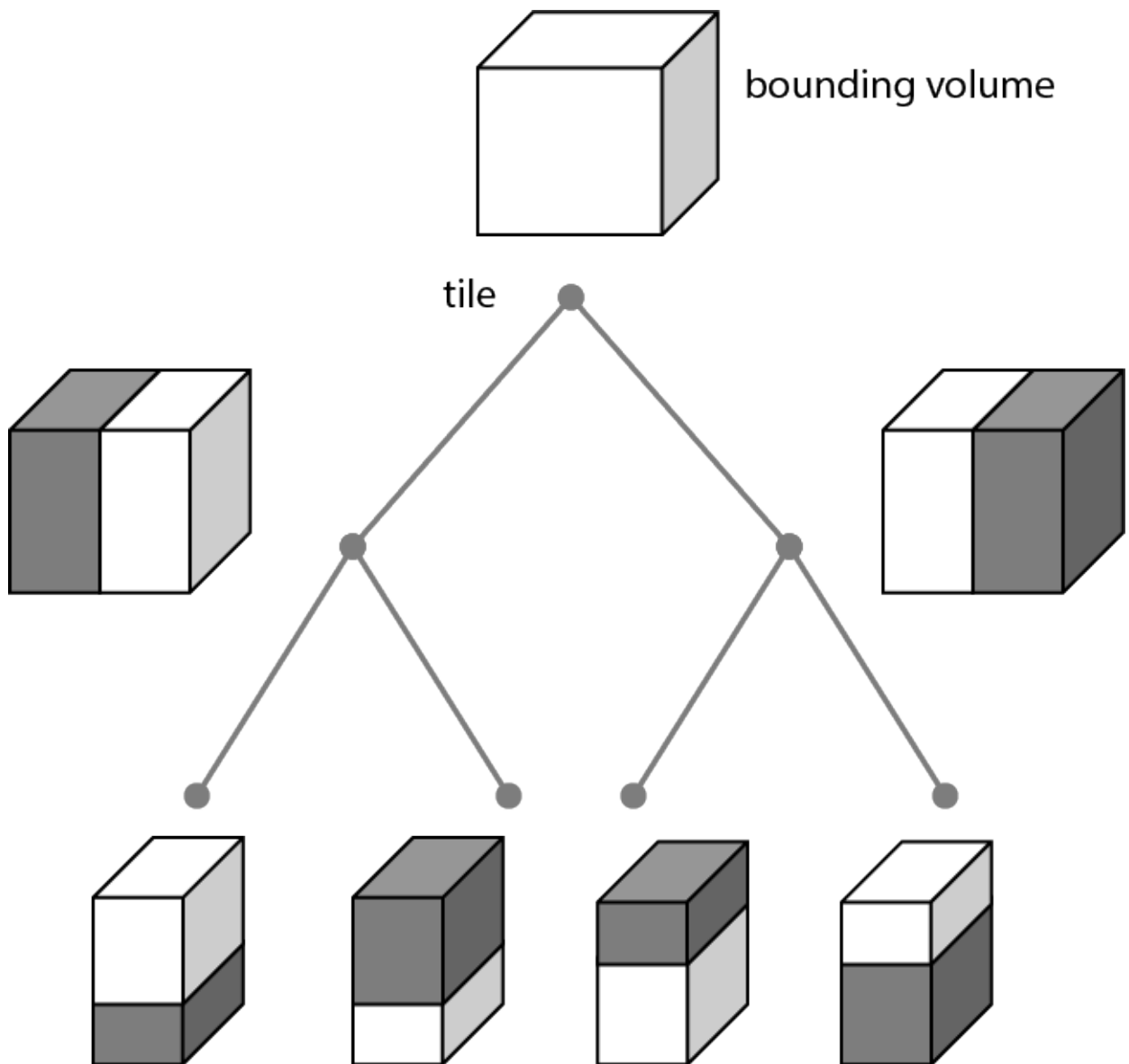


Figure A.1 – A tree of tiles.

A tileset may use a 2D spatial tiling scheme similar to raster and vector tiling schemes (like a Web Map Tile Service (WMTS) or XYZ scheme) that serve predefined tiles at several levels of detail (or zoom levels). However since the content of a tileset is often non-uniform or may not easily be organized in only two dimensions, the tree can be any spatial data structure with spatial coherence, including k-d trees, quadtrees, octrees, and grids. Implicit tiling defines a concise representation of quadtrees and octrees.

Application-specific *properties* may be provided at multiple granularities within a tileset. Properties may be associated with high-level entities like tilesets, tiles, contents, or features, or with individual vertices and texels. Properties conforms to a well-defined type system described by the [3D Metadata Specification](#), which may be extended with application- or domain-specific semantics.

Optionally a [3D Tiles Style](#), or *style*, may be applied to a tileset. A style defines expressions to be evaluated which modify how each feature is displayed.

A.9.2. Tileset

A tileset is a set of tiles organized in a spatial data structure, the tree. The tree has a single root tile. Each tile has zero or more children tiles. Tiles are represented as nodes in the glTF node hierarchy, or defined implicitly with [Implicit Tiling](#).

3D Tiles uses one main tileset file as the entry point to define a tileset. To create a tree of trees, a tileset may also reference [external tilesets](#).

The following example shows a tree with a root tile and a child tile.

```
{
  "extensionsUsed": ["3DTILES_tileset"],
  "extensionsRequired": ["3DTILES_tileset"],
  "asset": {
    "version": "2.1"
  },
  "scenes": [
    {
      "nodes": [0]
    }
  ],
  "scene": 0,
  "shapes": [
    {
      "type": "box",
      "box": {
        "size": [1.0, 1.0, 1.0]
      }
    },
    {
      "type": "box",
      "box": {
        "size": [0.9, 0.3, 1.0]
      }
    }
  ],
  "files": [
    {
      "mimeType": "model/gltf-binary",
      "uri": "root.glb"
    },
    {
      "mimeType": "model/gltf-binary",
      "uri": "child.glb"
    }
  ],
  "externalAssets": [
    {
```

```

    "file": 0
  },
  {
    "file": 1
  }
],
"extensions": {
  "3DTILES_tileset": {
    "geometricError": 240.0
  }
},
"nodes": [
  {
    "extensions": {
      "3DTILES_tileset": {
        "geometricError": 70.0,
        "refine": "ADD"
      }
    },
    "boundingVolume": {
      "shape": 0
    },
    "externalAsset": 0,
    "children": [1]
  },
  {
    "extensions": {
      "3DTILES_tileset": {
        "geometricError": 0.0
      }
    },
    "boundingVolume": {
      "shape": 1
    },
    "externalAsset": 1
  }
]
}

```

Listing A.1

The top-level 3DTILES_tileset extension has the following properties:

- `geometricError` is a nonnegative number that defines the error, in meters, that determines if the tileset is rendered. At runtime, the geometric error is used to compute *Screen-Space Error* (SSE), the error measured in pixels. If the SSE does not exceed a required minimum, the tileset should not be rendered, and none of its tiles should be considered for rendering. See [Geometric error](#).

Note: Tileset geometric error is different than tile geometric error. The former (which only appears once in the tileset) is used to determine whether the root tile is considered for rendering whereas the latter is used to determine whether a tile's children are considered for rendering.

Application-specific properties may be assigned to a tileset with [EXT_structural_metadata](#). See [Metadata](#).

A.9.3. Tile

Tiles are represented as nodes in the gITF node hierarchy. A tile consist of metadata used to determine if a tile is rendered, a reference to the renderable content, and an array of any children tiles.

tile

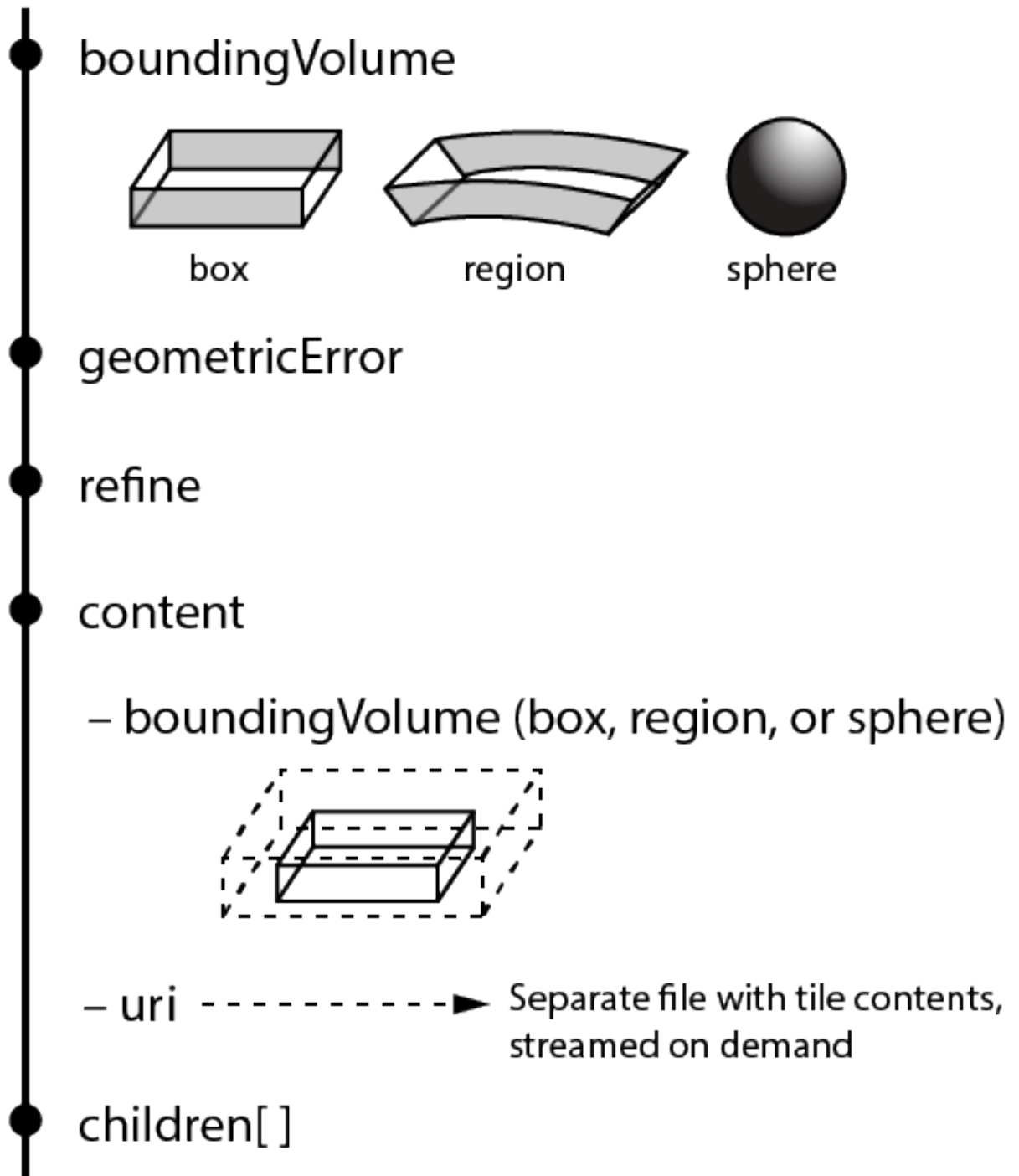


Figure A.2 — Elements of a tile JSON object

The following example shows the root tile from before:

```
{  
  "extensions": {  
    "3DTILES_tileset": {
```

```

        "geometricError": 70.0,
        "refine": "ADD"
    },
    "boundingVolume": {
        "shape": 0
    },
    "externalAsset": 0,
    "children": [1],
}

```

Listing A.2

The following glTF node properties contribute to the tile definition:

- `boundingVolume` defines a volume enclosing the tile, and is used to determine which tiles to render at runtime. See [Bounding Volumes](#) for the full list of supported shape types. The `boundingVolume` property **MUST** be provided when the node uses the `3DTILES_tileset` extension.
- `externalAsset` provides a reference to the tile's content. When `externalAsset` is not defined the tile is considered an *empty tile*. Empty tiles may be used for culling or other purposes.
- `children` is an array of node indices to child tiles. Each child tile's content is fully enclosed by its parent tile's `boundingVolume`. For *leaf tiles*, there are no children, and `children` **MUST** be omitted.
- `matrix` or translation, rotation, scale (not shown) define an optional local space transform for the tile, see [Transforms](#).

The `3DTILES_tileset` node extension has the following properties that further contribute to the tile definition:

- `geometricError` is a nonnegative number that defines the error, in meters, introduced if this tile is rendered and its children are not. At runtime, the geometric error is used to compute *Screen-Space Error* (SSE), the error measured in pixels. The SSE determines if a tile is sufficiently detailed for the current view or if its children should be considered. See [Geometric error](#).
- `refine` is a string that is either "REPLACE" for replacement refinement or "ADD" for additive refinement. It is required for the root tile of a tileset; it is optional for all other tiles. A tileset can use any combination of additive and replacement refinement. When the `refine` property is omitted, it is inherited from the parent tile. See [Refinement](#).
- `content` is an object that provides additional information about the tile's content, see [Content](#). If the tile does not have content (i.e. the node's `externalAsset` property is not defined) then `content` **MUST** be omitted.

The `mesh` property **MUST** be omitted.

Application-specific properties may be assigned to a tile with [EXT_structural_metadata](#). See [Metadata](#).

A.9.4. Content

A tile can be associated with renderable content, typically an external glTF asset that includes geometry and texture information, and may be extended to include metadata, model instancing, and compression.

The following example shows the root tile from before with a content object.

```
{
  "extensions": {
    "3DTILES_tileset": {
      "geometricError": 70.0,
      "refine": "ADD",
      "content": {
        "boundingVolume": {
          "shape": 2
        }
      }
    }
  },
  "boundingVolume": {
    "shape": 0
  },
  "externalAsset": 0,
  "children": [1],
}
```

Listing A.3

The content object includes the following optional properties:

- `boundingVolume` is a tightly fitting bounding volume enclosing just the tile's content. See [Bounding Volumes](#).

Application-specific properties may be assigned to content with [EXT_structural_metadata](#). See [Metadata](#).

A.9.5. External Tilesets

To create a tree of trees, a tile can point to an external tileset (a glTF using the `3DTILES_tileset` extension). This enables, for example, storing each city in a tileset and then having a global tileset of tilesets.

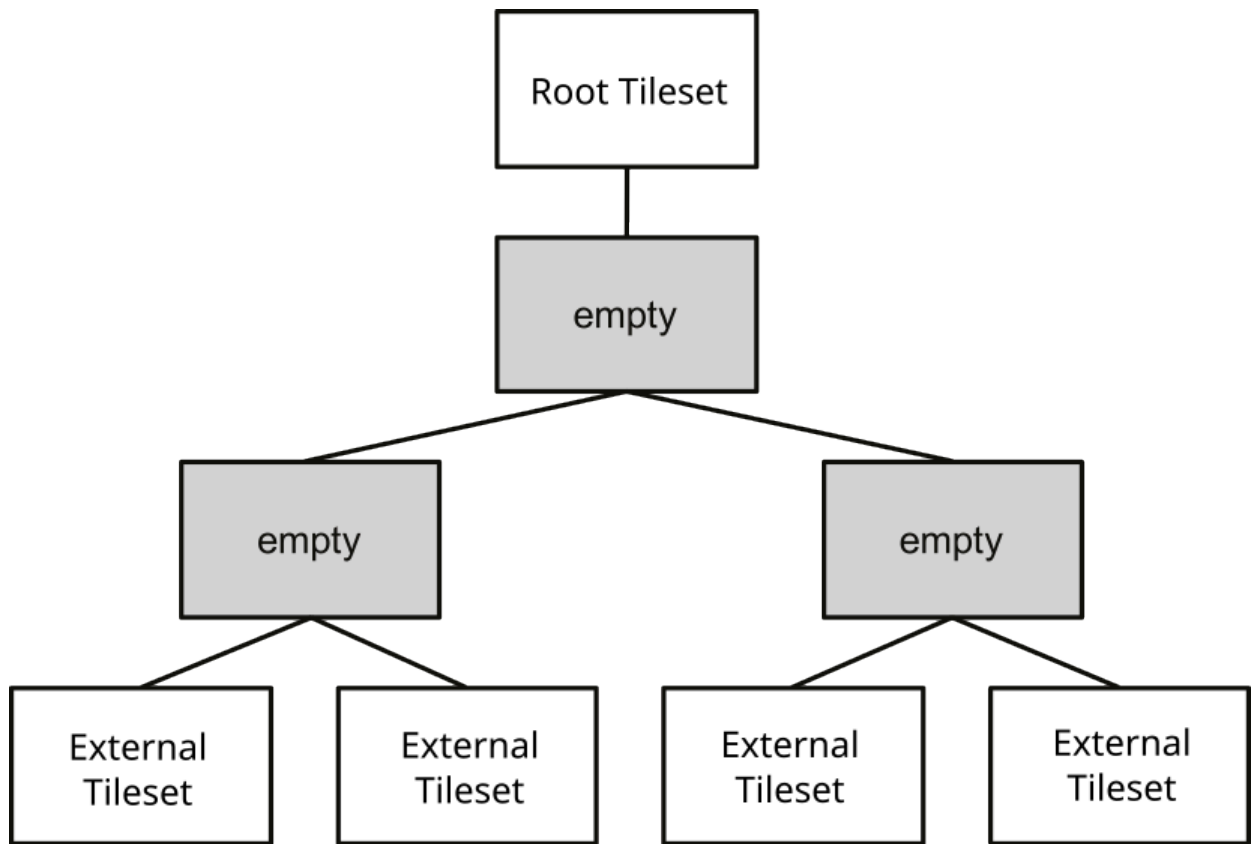


Figure A.3 — A tileset that refers to other tilesets

```

{
  "extensions": {
    "3DTILES_tileset": {
      "geometricError": 70.0,
      "refine": "ADD"
    }
  },
  "boundingVolume": {
    "shape": 0
  },
  "externalAsset": 0
}

```

Listing A.4

When a tile points to an external tileset, the tile:

- Cannot have any children; `node.children` **MUST** be omitted.
- Is unconditionally refinable regardless of its geometric error.

A.9.6. Geometric Error

Tiles are structured into a tree incorporating *Hierarchical Level of Detail* (HLOD) so that at runtime a client implementation will need to determine if a tile is sufficiently detailed for

rendering and if the content of tiles should be successively refined by children tiles of higher resolution. An implementation will consider a maximum allowed *Screen-Space Error* (SSE), the error measured in pixels.

A tile's geometric error defines the selection metric for that tile. Its value is a nonnegative number that defines the error, in meters, introduced if this tile is rendered and its children are not.

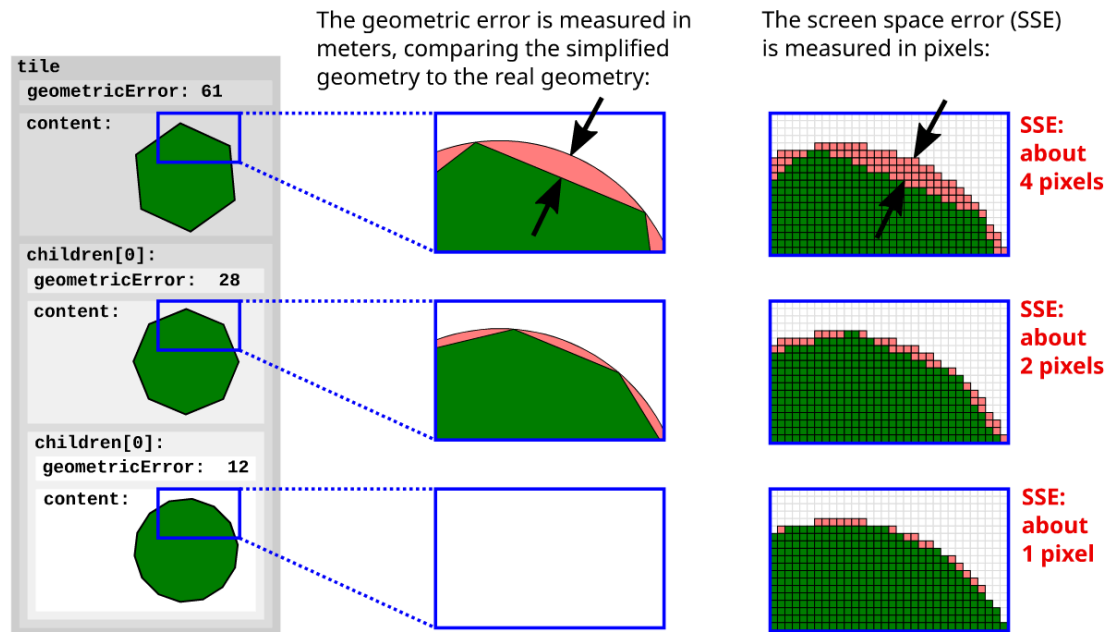


Figure A.4 — Geometric Error and Screen Space Error (SSE)

Generally, the root tile will have the largest geometric error, and each successive level of children will have a smaller geometric error than its parent, with leaf tiles having a geometric error of or close to 0. If a child tile's geometric error is greater than or equal to its parent's geometric error, that child tile is considered unconditionally refinable.

In a client implementation, geometric error is used with other screen space metrics—e.g., distance from the tile to the camera, screen size, and resolution—to calculate the SSE introduced if this tile is rendered and its children are not. If the introduced SSE exceeds the maximum allowed, then the tile is refined and its children are considered for rendering.

The geometric error is formulated based on a metric like point density, mesh or texture decimation, or another factor specific to that tileset. In general, a higher geometric error means a tile will refine more aggressively, and children tiles will be loaded and rendered sooner.

A.9.7. Refinement

Refinement determines the process by which a lower resolution parent tile renders when its higher resolution children are selected to be rendered. Permitted refinement types are replacement ("REPLACE") and additive ("ADD"). If the tile has replacement refinement, the

children tiles are rendered in place of the parent, that is, the parent tile is no longer rendered. If the tile has additive refinement, the children are rendered in addition to the parent tile.

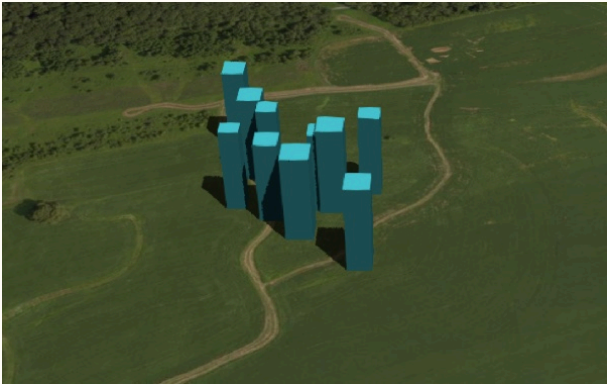
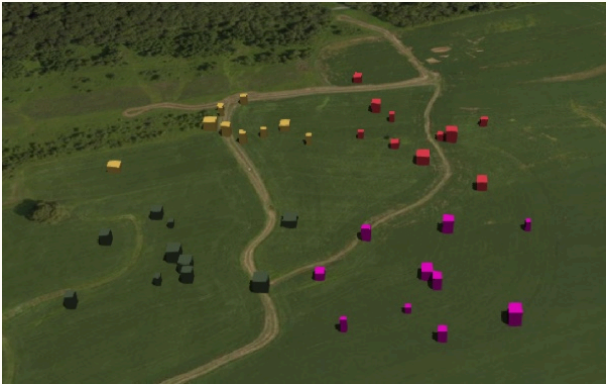
A tileset can use replacement refinement exclusively, additive refinement exclusively, or any combination of additive and replacement refinement.

A refinement type is required for the root tile of a tileset; it is optional for all other tiles. When omitted, a tile inherits the refinement type of its parent.

A.9.7.1. Replacement

If a tile uses replacement refinement, when refined it renders its children in place of itself.

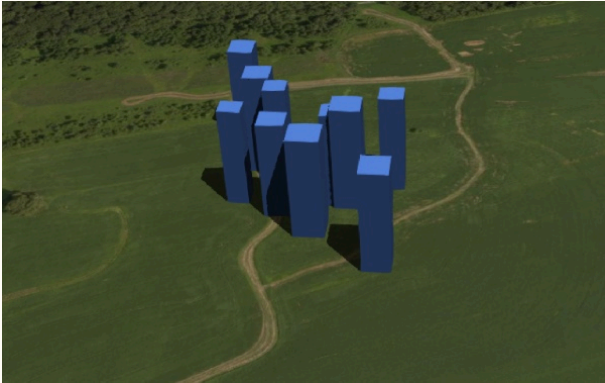
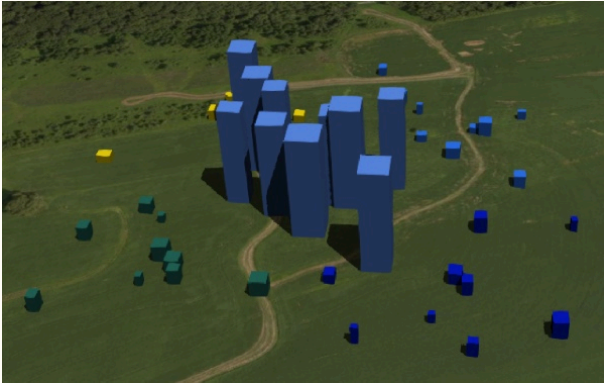
Table A.1 — A tile and a refined tile using replacement refinement

PARENT TILE	REFINED
	
Figure A.5	Figure A.6

A.9.7.2. Additive

If a tile uses additive refinement, when refined it renders itself and its children simultaneously.

Table A.2 — A tile and a refined tile using additive refinement

PARENT TILE	REFINED
	
Figure A.7	Figure A.8

A.9.7.3. Unconditional Refinement

A tile that is “unconditionally refined” will always be REFINED, it will never be RENDERED. Such tiles are identified by having a geometric error greater than or equal to its parent’s geometric error.

One example of an unconditionally refined tile is a tile referencing an [external tileset](#). This tile has no renderable content itself, but may have child tiles with renderable content.

A.9.8. Bounding Volumes

A bounding volume defines the spatial extent enclosing a tile or a tile’s content. The bounding volume shape type **MUST** be "box" or "sphere" unless additional shape types are enabled through extensions.

A list of extensions that enable additional shape types:

- [3DTILES_shape_ellipsoid_region](#)
- [3DTILES_shape_cylinder_region](#)
- [3DTILES_shape_s2](#)

The content may have an optional boundingVolume, the content bounding volume. Unlike the tile bounding volume, the content bounding volume is a tightly fitting bounding volume enclosing just the tile’s content. This enables tight view frustum culling, excluding from rendering any content not in the volume of what is potentially in view. When it is not defined, the tile’s bounding volume is still used for culling.

The screenshot below shows the bounding volumes for the root tile for Canary Wharf. The node.boundingBox, shown in red, encloses the entire area of the tileset; content.boundingBox shown in blue, encloses just the four features (models) in the root tile.

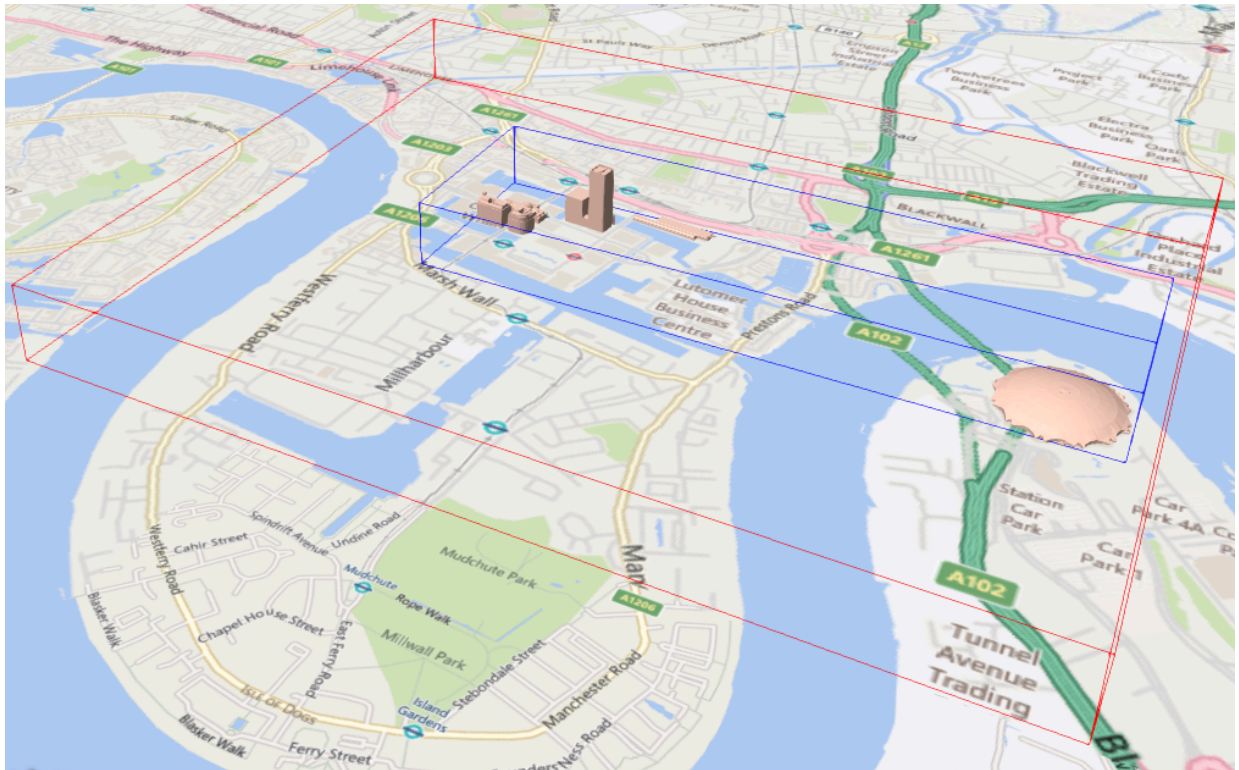


Figure A.9 — Bounding volumes for the root tile of a tileset. Building data from [CyberCity3D](#). Imagery data from [Bing Maps](#).

A.9.8.1. Bounding Box

The following example shows an oriented bounding box that is created by transforming an axis-aligned box shape.

```
{
  "shapes": [
    {
      "type": "box",
      "box": {
        "size": [1.0, 1.0, 1.0]
      }
    }
  ],
  "nodes": [
    {
      "boundingVolume": {
        "shape": 0,
        "translation": [-3923021.73, -931070.70, 4925458.17],
        "rotation": [0.26262, -0.20758, -0.58433, 0.73925],
        "scale": [100.0, 100.0, 100.0]
      },
      ...
    }
  ]
}
```

```

    ]
  }

```

Listing A.5

A.9.8.2. Bounding Sphere

The following example shows a bounding sphere.

```

{
  "shapes": [
    {
      "type": "sphere",
      "sphere": {
        "radius": 6378137.0
      }
    }
  ],
  "nodes": [
    {
      "boundingVolume": {
        "shape": 0
      },
      ...
    }
  ]
}

```

Listing A.6

A.9.9. Transforms

A tile may define a local space transform using the matrix or translation, rotation, scale glTF node properties.

The transformation from each tile's local coordinate system to the tileset's global coordinate system is computed by a top-down traversal of the tileset and by post-multiplying a child's transform with its parent's transform, as covered in [glTF Transformations](#).

The transform applies to the tile's bounding volume, content (if present), and content bounding volume (if present).

The transform does not apply to `geometricError`—i.e., the scale defined by transform does not scale the geometric error—the geometric error is always defined in meters.

Certain bounding volume types, such as `3DTILES_shape_ellipsoid_region` and `3DTILES_shape_s2`, are defined in a geospatial coordinate system and cannot be reasonably transformed. In such cases, the transform does not apply to these bounding volumes. Additionally, the transform does not apply to `3DTILES_horizon_occlusion_point`.

[EXT_georeference](#) may be used to transform a tile from its local coordinate system to a geocentric coordinate system by placing it at a specific longitude, latitude, height. The georeference transform is applied after the node transform (pre-multiplied).

A.9.10. Spatial Coherence

As described above, the tree has spatial coherence; each tile has a bounding volume completely enclosing its content, and the content for child tiles are completely inside the parent's bounding volume. This does not imply that a child's bounding volume is completely inside its parent's bounding volume. For example:

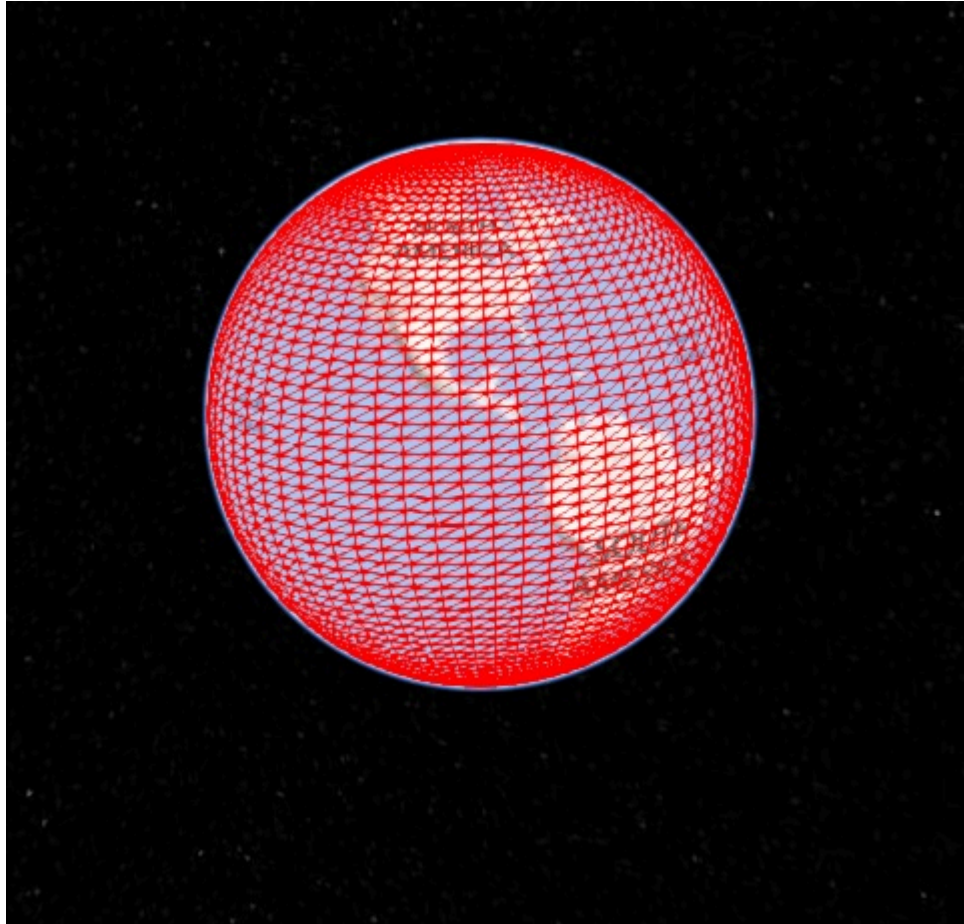


Figure A.10 — Bounding sphere for a terrain tile.

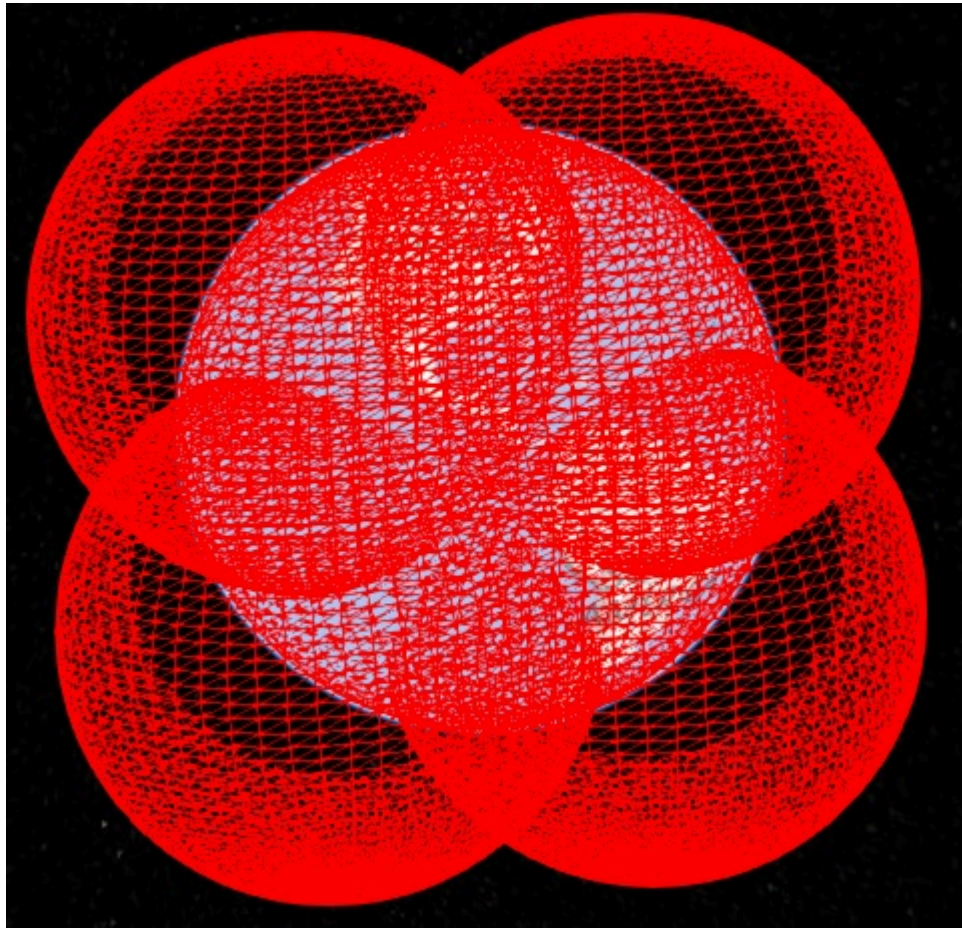


Figure A.11 — Bounding spheres for the four child tiles. The children's content is completely inside the parent's bounding volume, but the children's bounding volumes are not since they are not tightly fit.

A.9.11. Spatial Data Structures

3D Tiles incorporates the concept of Hierarchical Level of Detail (HLOD) for optimal rendering of spatial data. A tileset is composed of a tree, defined by a root tile and, recursively, its children tiles, which can be organized by different types of spatial data structures.

A runtime engine is generic and will render any tree defined by a tileset. Any combination of tile formats and refinement approaches can be used, enabling flexibility in supporting heterogeneous datasets, see [Refinement](#).

A tileset may use a 2D spatial tiling scheme similar to raster and vector tiling schemes (like a Web Map Tile Service (WMTS) or XYZ scheme) that serve predefined tiles at several levels of detail (or zoom levels). However since the content of a tileset is often non-uniform or may not easily be organized in only two dimensions, other spatial data structures may be more optimal.

[Appendix A: Spatial data structures](#) gives a brief description of how 3D Tiles can represent various spatial data structures.

A.9.12. Coordinate Reference System (CRS)

A tileset may be defined in either a **global** or **local** coordinate system. A tileset's global coordinate system will often be in a WGS 84 Earth-centered, Earth-fixed (ECEF) reference frame (EPSG 4978), but it doesn't have to be, e.g., a power plant may be defined fully in its local coordinate system.

A tileset defined in a local coordinate system follows the standard glTF coordinate system conventions: right-handed, +Y up, and linear units in meters. It **MUST NOT** use the EXT_geospatial_crs extension.

A tileset defined in a global coordinate system **MUST** specify its coordinate reference system (CRS) with EXT_geospatial_crs.

A tileset defined in a global coordinate system differs from the standard glTF coordinate system conventions – there is no single “up” vector on the surface of a globe. Instead, the tileset uses a **geocentric (planetocentric)** coordinate reference system such as EPSG 4978.

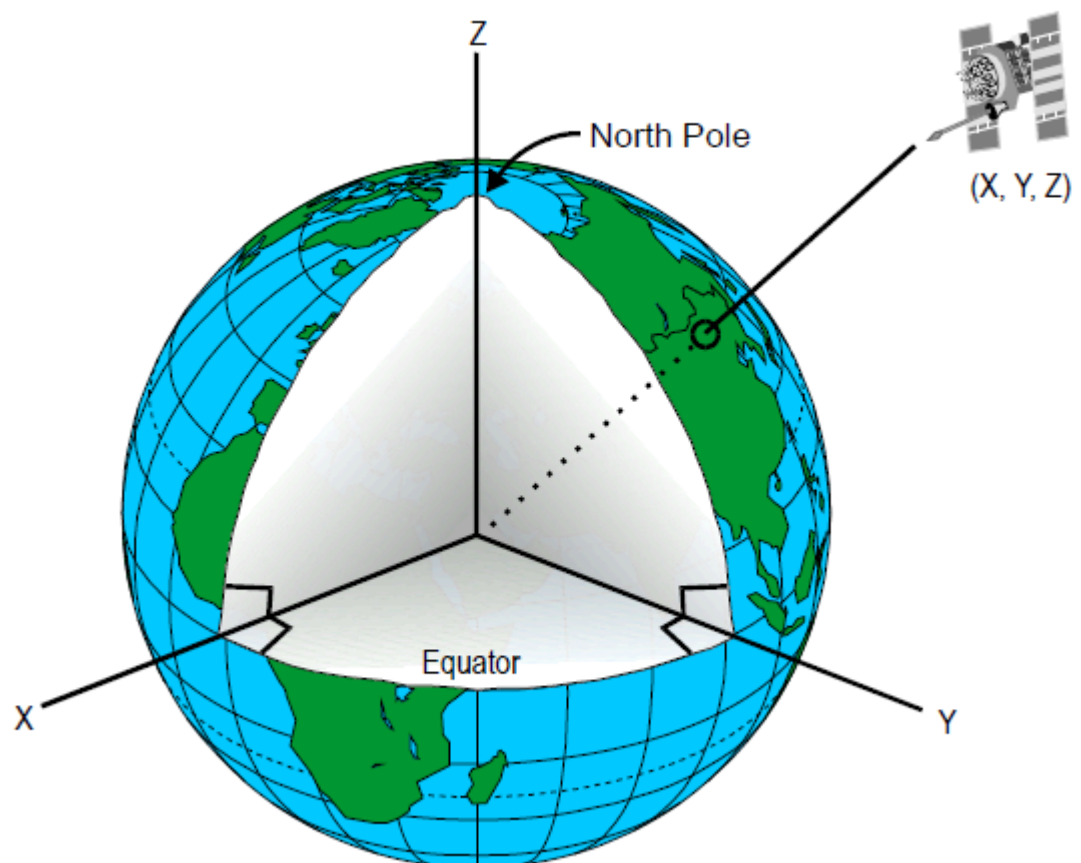


Figure A.12 — An illustration of EPSG 4978, an Earth-centered, Earth-fixed (ECEF) geocentric coordinate reference system. It defines 0,0,0 as the center of mass on Earth, where +Z extends through true north (i.e. the geodetic North Pole) and +X intersects the sphere of

the earth at 0° latitude (the equator) and 0° longitude (the prime meridian which passes through Greenwich). As a result, no discrete "Up" vector exists relative to a ground plane.

The example below shows a tileset using [EPSG 4978](#).

```
{
  "asset": {
    "version": "2.1"
  },
  "extensions": {
    "EXT_geospatial_crs": {
      "format": "wkid",
      "extensions": {
        "EXT_geospatial_crs_wkid": {
          "authority": "EPSG",
          "wkid": 4978
        }
      }
    }
  }
}
```

Listing A.7

NOTE: 3D Tiles only allows local and geocentric (planetocentric) coordinate reference systems. Other coordinate reference system types, such as geographic and projected coordinate reference systems, are not allowed as they often require dedicated coordinate transformation libraries and ancillary data, such as grid shift files, in order to be rendered in 3D globe engines.

Tilesets may reference [external tilesets](#) in different coordinate systems. For example, a tileset could start in a geocentric coordinate reference system and then transition to a local engineering reference frame for higher precision.

The following rules apply for CRS transitions:

- Tilesets in a local coordinate system **MUST** only reference other tilesets in a local coordinate system
- Tilesets in a geocentric coordinate system **MUST** only reference tilesets in a local coordinate system or tilesets in the same geocentric coordinate system.

A [tile transform](#) may be applied to transform a tile's local coordinate system to the parent tile's geocentric coordinate system.

The example below shows a tileset using [EXT_georeference](#). Note that when [EXT_georeference](#) is used the tileset is in a global coordinate system and [EXT_geospatial_crs](#) must also be used.

```
{
  "asset": {
    "version": "2.1"
  },
  "extensions": {
    "EXT_geospatial_crs": {
      "format": "wkid",
      "extensions": {
        "EXT_geospatial_crs_wkid": {
          "authority": "EPSG",

```

```

        "wkid": 4978
      }
    }
  },
  "nodes": [
    {
      "extensions": {
        "3DTILES_tileset": {
          "geometricError": 240.0,
          "refine": "ADD"
        },
        "EXT_georeference": {
          "longitude": -75.15836368768382,
          "latitude": 39.95090650840344,
          "height": -21.668226434267066
        }
      },
      "boundingVolume": {
        "shape": 0
      },
      "externalAsset": 0
    }
  ]
}

```

Listing A.8

Certain bounding volume types, such as 3DTILES_shape_ellipsoid_region and 3DTILES_shape_s2, are defined in a geospatial coordinate system and **CANNOT** be used by tiles defined in a local coordinate system.

Additionally, 3DTILES_horizon_occlusion_point **CANNOT** be used by tiles defined in a local coordinate system.

A.9.13. Implicit Tiling

The bounding volume hierarchy may be defined explicitly — as shown previously — which enables a wide variety of spatial data structures. Certain common data structures such as quadtrees and octrees may be defined implicitly without providing bounding volumes for every tile. This regular pattern allows for random access of tiles based on their tile coordinates which enables accelerated spatial queries, new traversal algorithms, and efficient updates of tile content, among other use cases.

Implicit tiling is enabled by using the [3DTILES_implicit_tiling](#) extension. In order to support sparse datasets, the tree is partitioned into fixed-size *subtrees*. See [3DTILES_subtree](#).

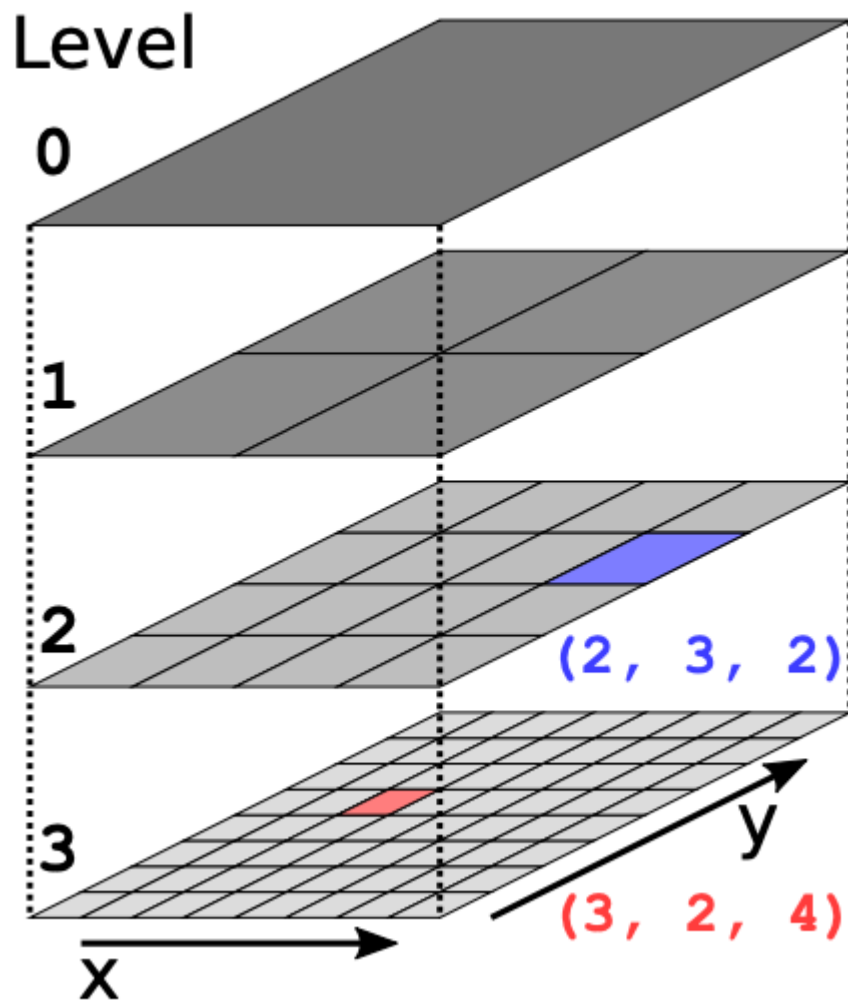


Figure A.13 – Quadtree with tile coordinates

A.10. Supporting Extensions

A.10.1. Metadata

Application-specific *properties* may be provided at multiple granularities within a tileset. Properties may be associated with tilesets, tiles, and contents within a tileset file using the `EXT_structural_metadata` extension. Properties may also be associated with features or with individual vertices and texels within content files.

The following example shows a tileset with tileset properties, tile properties, and content properties. The referenced asset root.glb may also use EXT_structural_metadata, for example, for storing per-feature properties.

```
{
  "extensionsUsed": ["3DTILES_tileset", "EXT_structural_metadata"],
  "extensionsRequired": ["3DTILES_tileset"],
  "extensions": {
    "3DTILES_tileset": {
      "geometricError": 240,
      "extensions": {
        "EXT_structural_metadata": {
          "class": "city",
          "properties": {
            "name": "New York City",
            "country": "United States",
            "population": 8804190
          }
        }
      }
    }
  },
  "EXT_structural_metadata": {
    "schema": {
      "id": "schema",
      "classes": {
        "city": {
          "properties": {
            "name": {
              "type": "STRING"
            },
            "country": {
              "type": "STRING"
            },
            "population": {
              "type": "SCALAR",
              "componentType": "UINT64"
            }
          }
        }
      }
    },
    "block": {
      "properties": {
        "borough": {
          "type": "STRING"
        },
        "zipCode": {
          "type": "SCALAR",
          "componentType": "UINT32"
        },
        "population": {
          "type": "SCALAR",
          "componentType": "UINT64"
        }
      }
    },
    "geometryData": {
      "properties": {
        "vertices": {
          "type": "SCALAR",
          "componentType": "UINT32"
        },
        "primitives": {
          "type": "SCALAR",

```

```

        "componentType": "UINT32"
      }
    }
  }
},
"asset": {
  "version": "2.1"
},
"nodes": [
  {
    "extensions": {
      "3DTILES_tileset": {
        "geometricError": 0.0,
        "refine": "ADD",
        "content": {
          "extensions": {
            "EXT_structural_metadata": {
              "class": "geometryData",
              "properties": {
                "vertices": 49534,
                "primitives": 2
              }
            }
          }
        }
      },
      "EXT_structural_metadata": {
        "class": "block",
        "properties": {
          "borough": "Manhattan",
          "zipCode": 10024,
          "population": 52428
        }
      }
    },
    "boundingVolume": {
      "shape": 0
    },
    "externalAsset": 0
  }
]
}

```

Listing A.9

A.10.1.1. Statistics

A tileset may also provide summary statistics with `EXT_structural_metadata` that allow applications to analyze or display metadata—e.g. with [declarative styling](#)—without first having to process the complete dataset to identify bounds for color ramps and histograms. Statistics are provided on a per-class basis, so that applications can provide styling or context based on the tileset as a whole, while only needing to download and process a subset of its tiles.

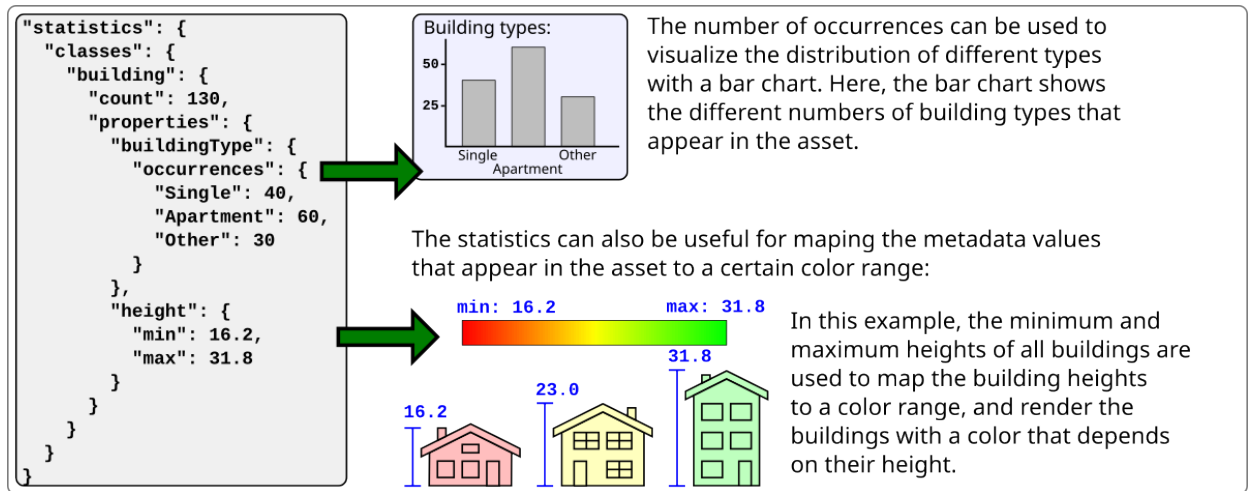


Figure A.14 – Metadata statistics using *EXT_structural_metadata*

A.10.2. Layers

Layers are a common concept when working with geospatial data, representing semantic or functional groups of geometries requiring common handling by the application. The [3DTILES_layers](#) extension adds support for layers. This can be useful for a variety of applications including vector data. See the [3DTILES_layers](#) extension specification for further details.

A.10.3. Horizon Occlusion Point

To allow for fast horizon culling at runtime, the [3DTILES_horizon_occlusion_point](#) extension can be used to set a precomputed horizon occlusion point to a tile or contents. See the [3DTILES_horizon_occlusion_point](#) extension specification for further details.

A.10.4. Visibility Bounding Volumes

Visibility bounding volumes allow specifying a space where specialized geometry is only visible when a camera is present. The [EXT_node_visibility_volume](#) extension allows specifying a bounding volume where an associated node containing a [KHR_node_visibility](#) definition is set to become visible. See the [EXT_node_visibility_volume](#) extension specification for further details.

A.10.5. Declarative Styling

3D Tiles includes concise declarative styling defined with JSON and expressions written in a small subset of JavaScript augmented for styling.

For complete details, see the [Declarative Styling](#) specification.

A.11. Appendix A: Spatial Data Structures

A.11.1. Quadtrees

A quadtree is created when each tile has four uniformly subdivided children, similar to typical 2D geospatial tiling schemes. Empty child tiles can be omitted.

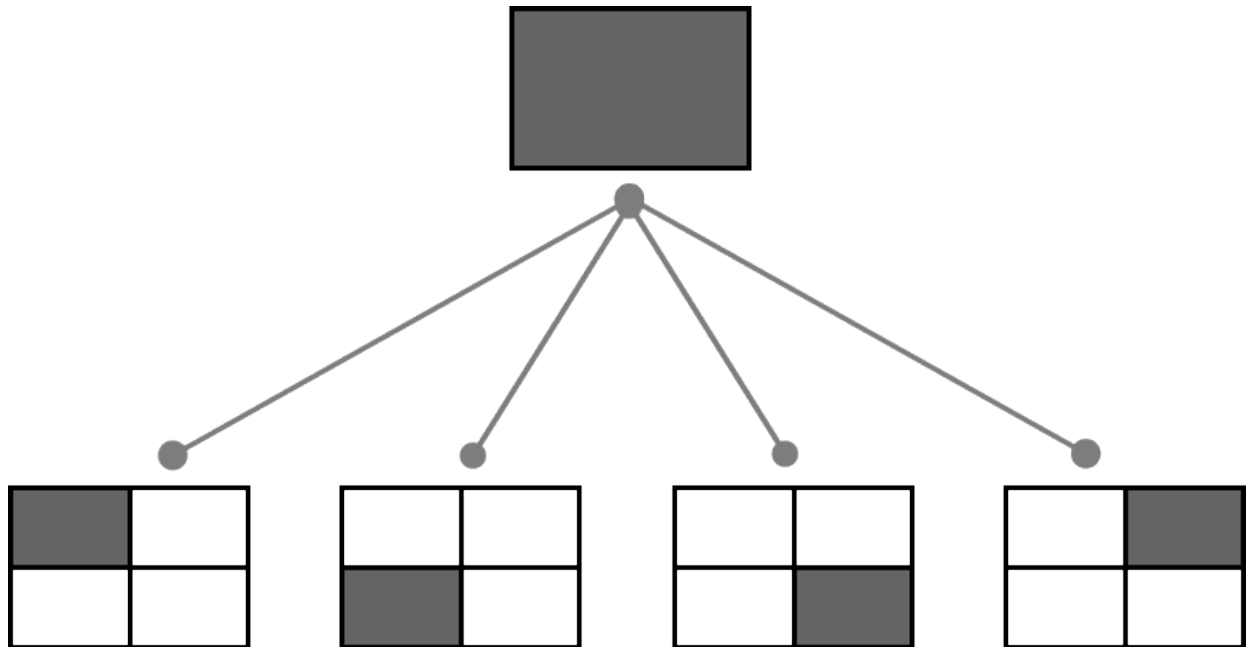


Figure A.15 – Classic quadtree subdivision.

3D Tiles enable quadtree variations such as non-uniform subdivision and tight bounding volumes (as opposed to bounding, for example, the full 25% of the parent tile, which is wasteful for sparse datasets).

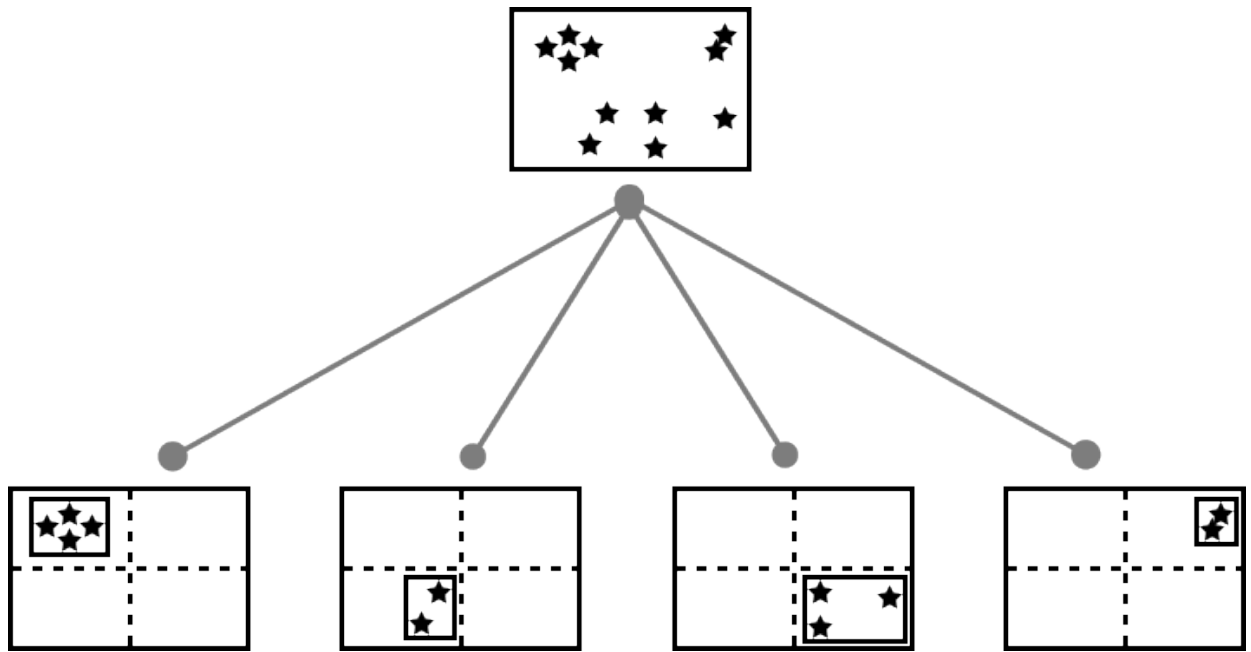


Figure A.16 – Quadtree with tight bounding volumes around each child.

For example, here is the root tile and its children for Canary Wharf. Note the bottom left, where the bounding volume does not include the water on the left where no buildings will appear:

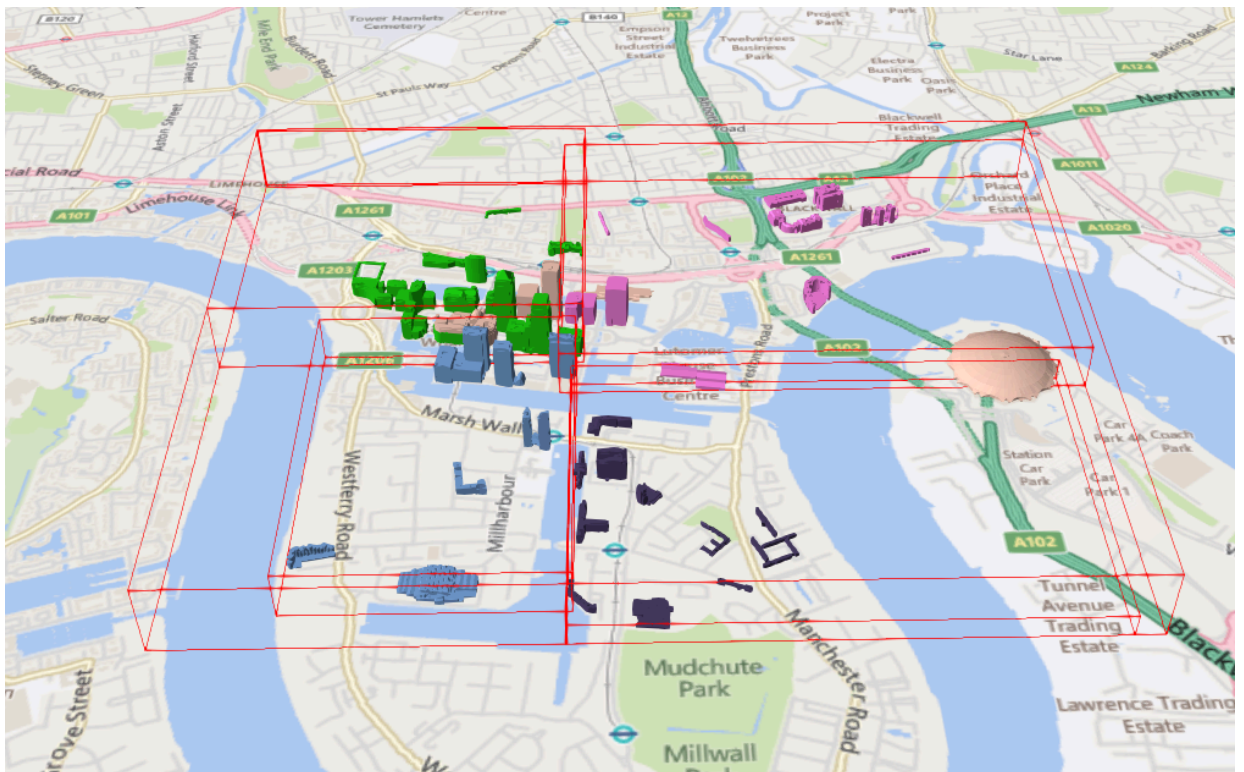


Figure A.17 – Building data from [CyberCity3D](#). Imagery data from [Bing Maps](#).

3D Tiles also enable other quadtree variations such as loose quadtrees, where child tiles overlap but spatial coherence is still preserved, i.e., a parent tile completely encloses all of its children. This approach can be useful to avoid splitting features, such as 3D models, across tiles.

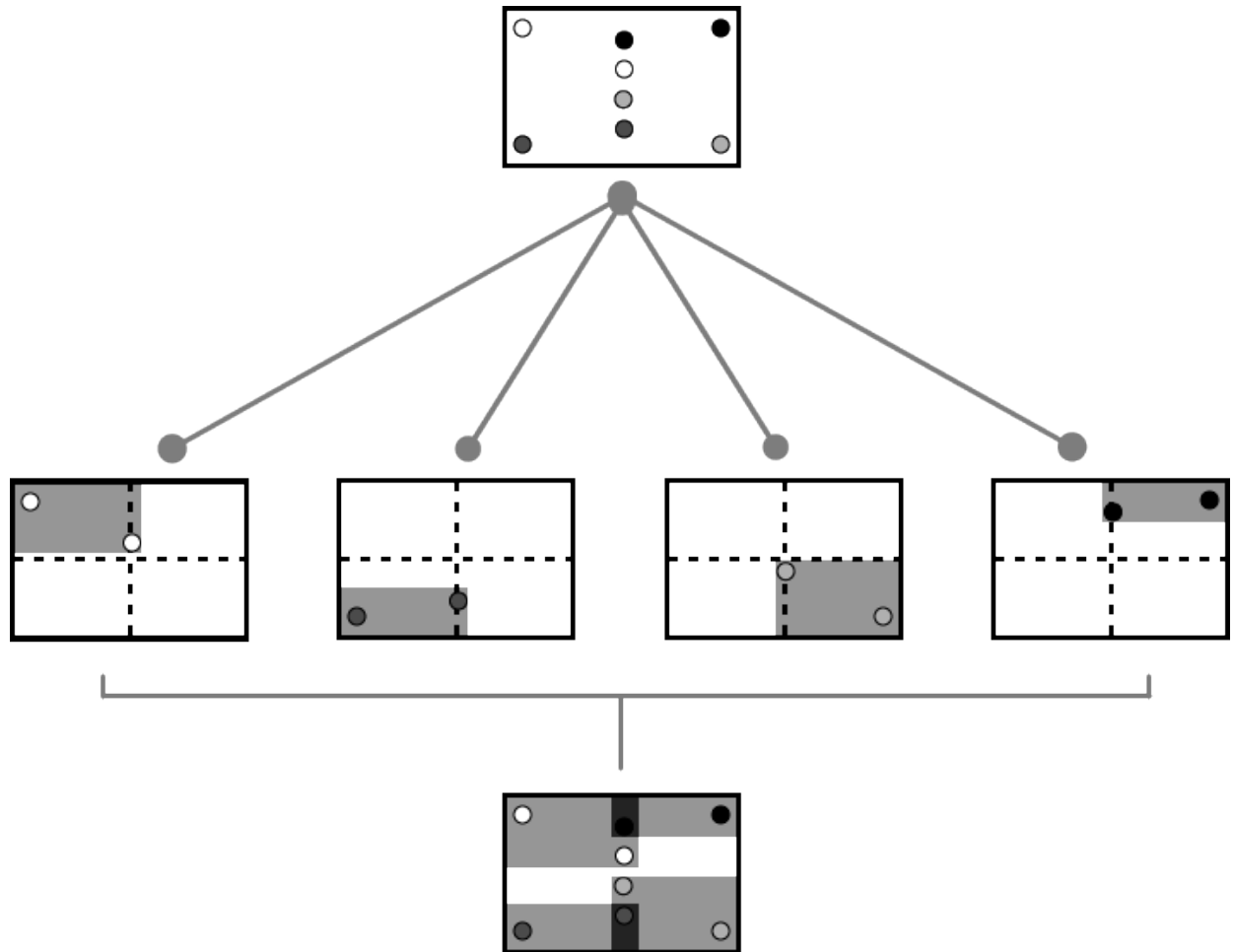


Figure A.18 – Quadtree with non-uniform and overlapping tiles.

Below, the green buildings are in the left child and the purple buildings are in the right child. Note that the tiles overlap so the two green and one purple building in the center are not split.

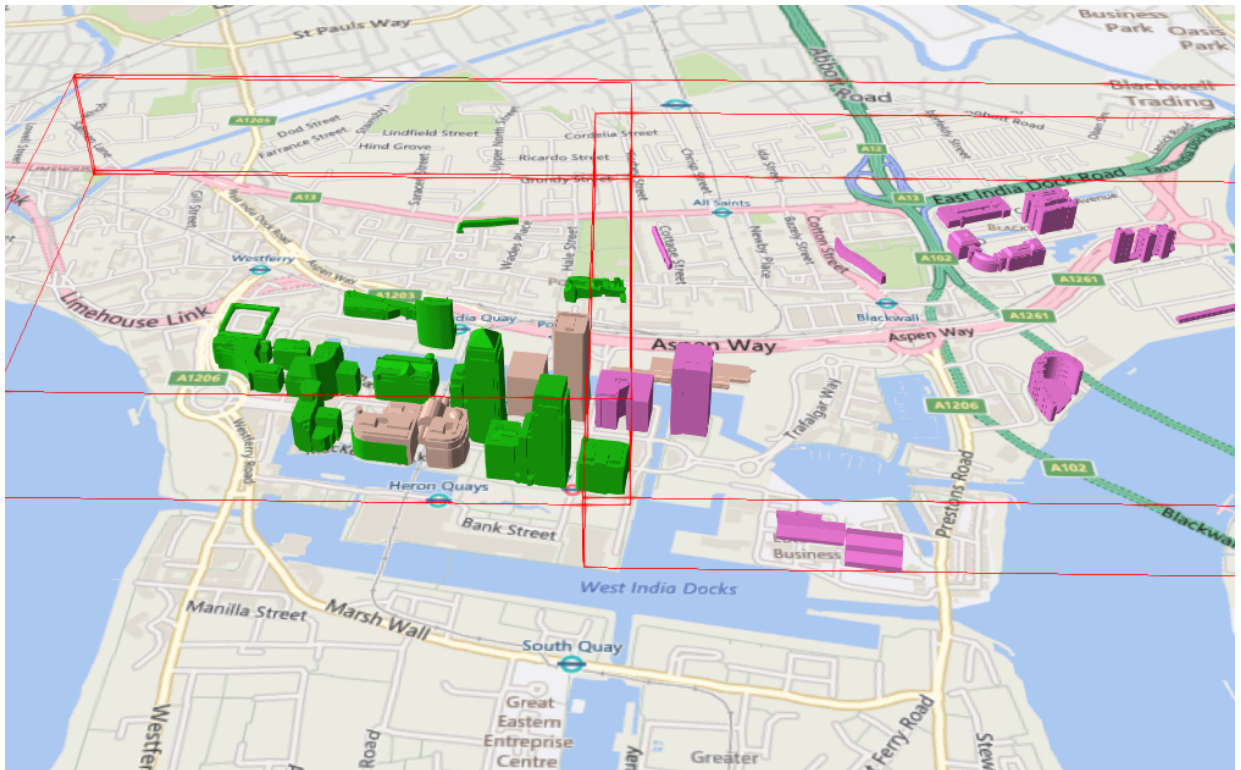


Figure A.19 — Building data from [CyberCity3D](#). Imagery data from [Bing Maps](#).

A.11.1.1. K-d trees

A k-d tree is created when each tile has two children separated by a *splitting plane* parallel to the x, y, or z axis (or latitude, longitude, height). The split axis is often round-robin rotated as levels increase down the tree, and the splitting plane may be selected using the median split, surface area heuristics, or other approaches.

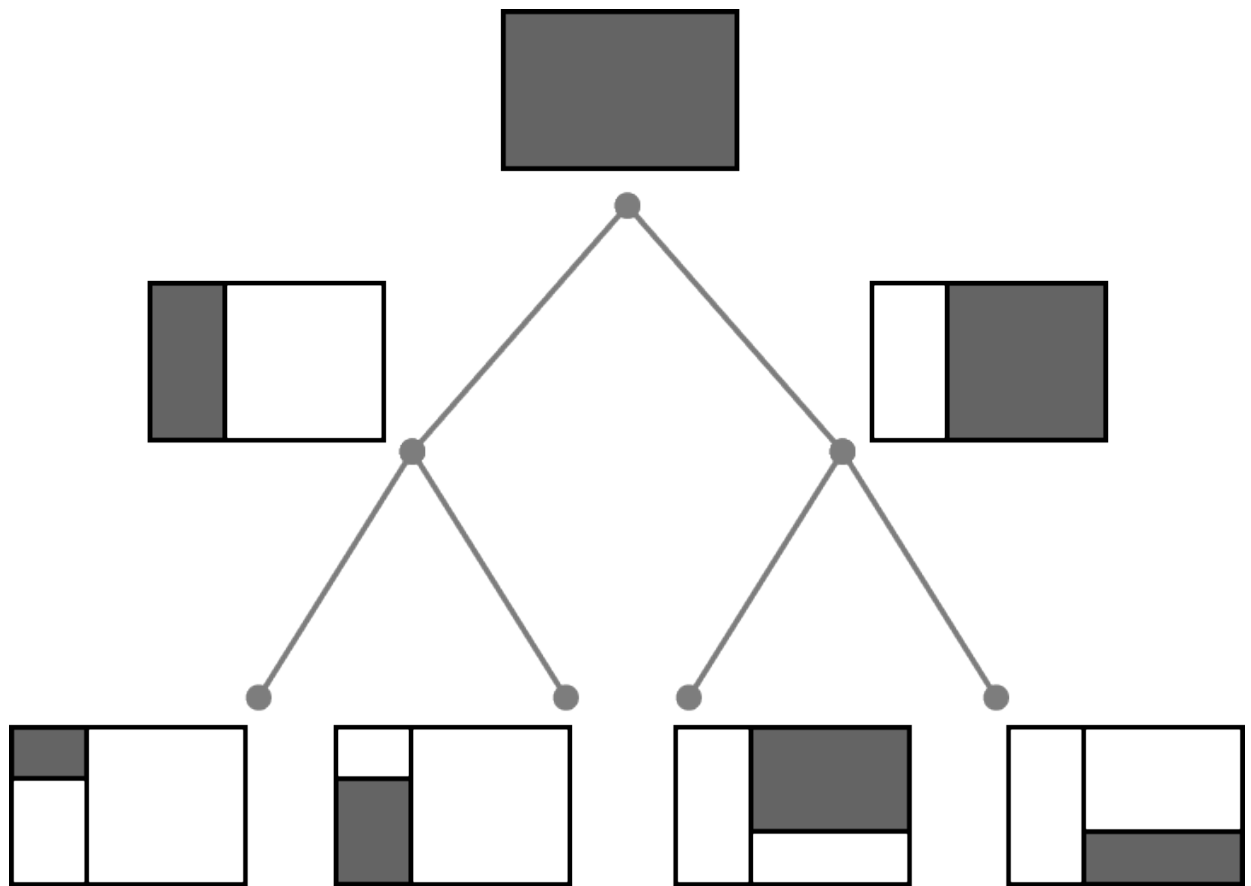


Figure A.20 – Example k-d tree. Note the non-uniform subdivision.

Note that a k-d tree does not have uniform subdivision like typical 2D geospatial tiling schemes and, therefore, can create a more balanced tree for sparse and non-uniformly distributed datasets.

3D Tiles enables variations on k-d trees such as multi-way k-d trees where, at each leaf of the tree, there are multiple splits along an axis. Instead of having two children per tile, there are n children.

A.11.1.2. Octrees

An octree extends a quadtree by using three orthogonal splitting planes to subdivide a tile into eight children. Like quadtrees, 3D Tiles allows variations to octrees such as non-uniform subdivision, tight bounding volumes, and overlapping children.

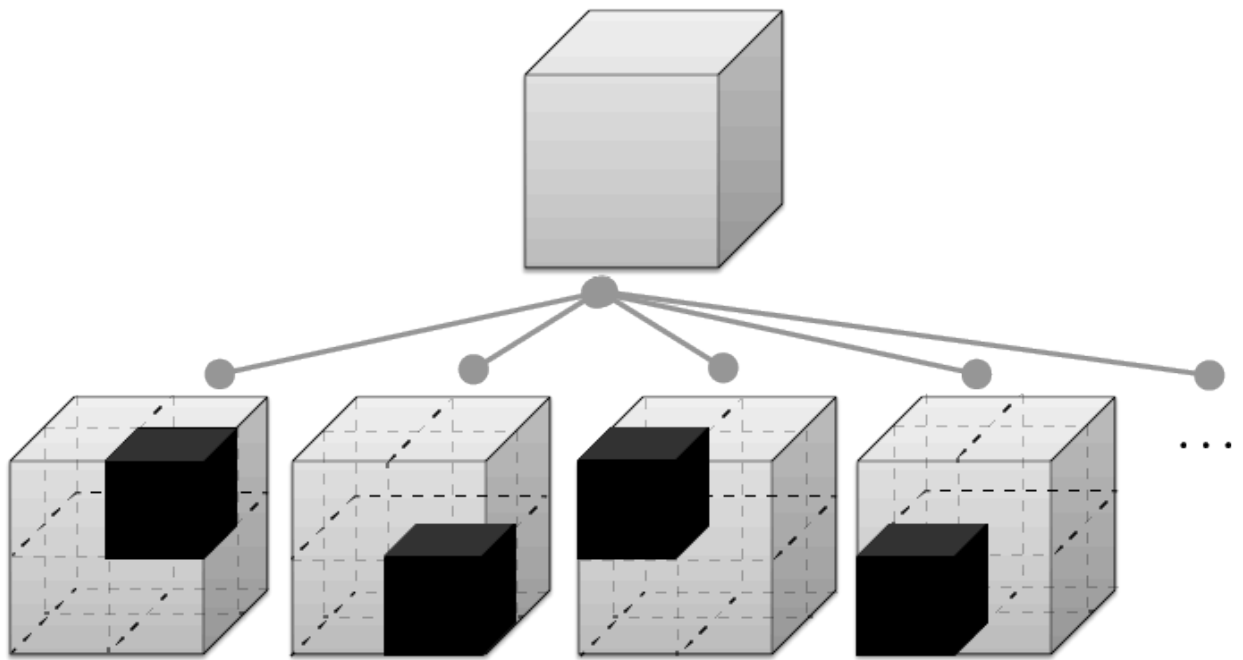


Figure A.21 – Traditional octree subdivision.

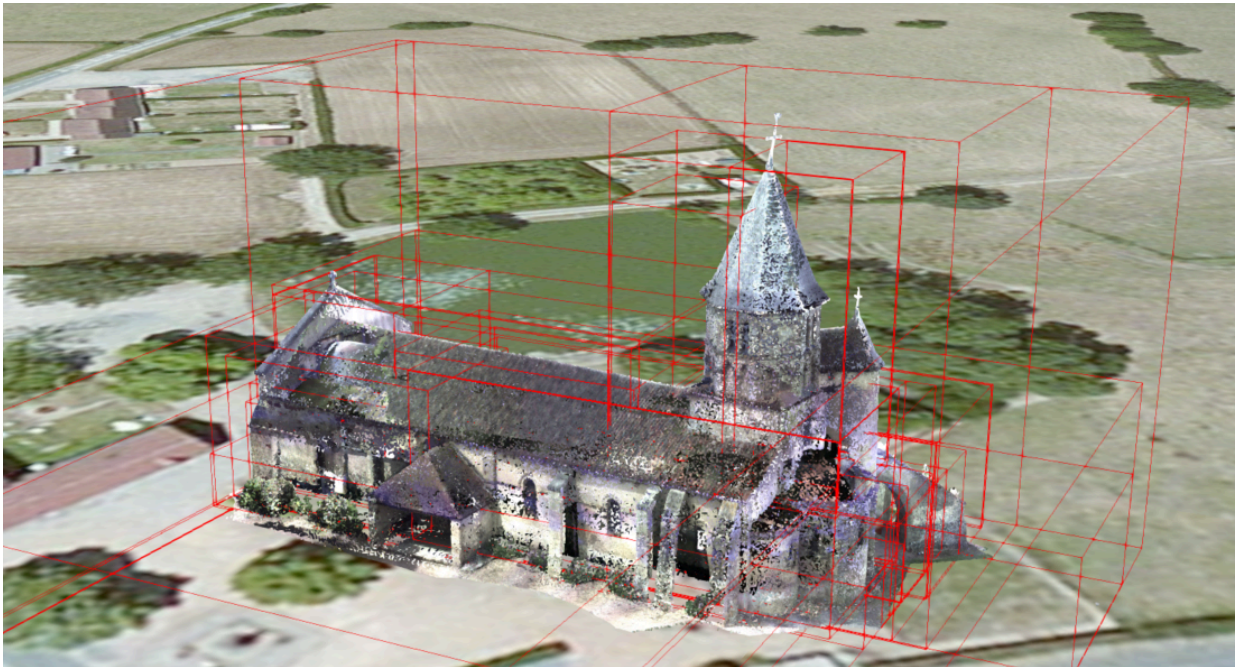


Figure A.22 – Non-uniform octree subdivision for a point cloud using additive refinement. Point Cloud of [the Church of St Marie at Chappes, France](#) by Prof. Peter Allen, Columbia University Robotics Lab. Scanning by Alejandro Troccoli and Matei Ciocarlie.

A.11.1.3. Grids

3D Tiles enables uniform, non-uniform, and overlapping grids by supporting an arbitrary number of child tiles. For example, here is a top-down view of a non-uniform overlapping grid of Cambridge:

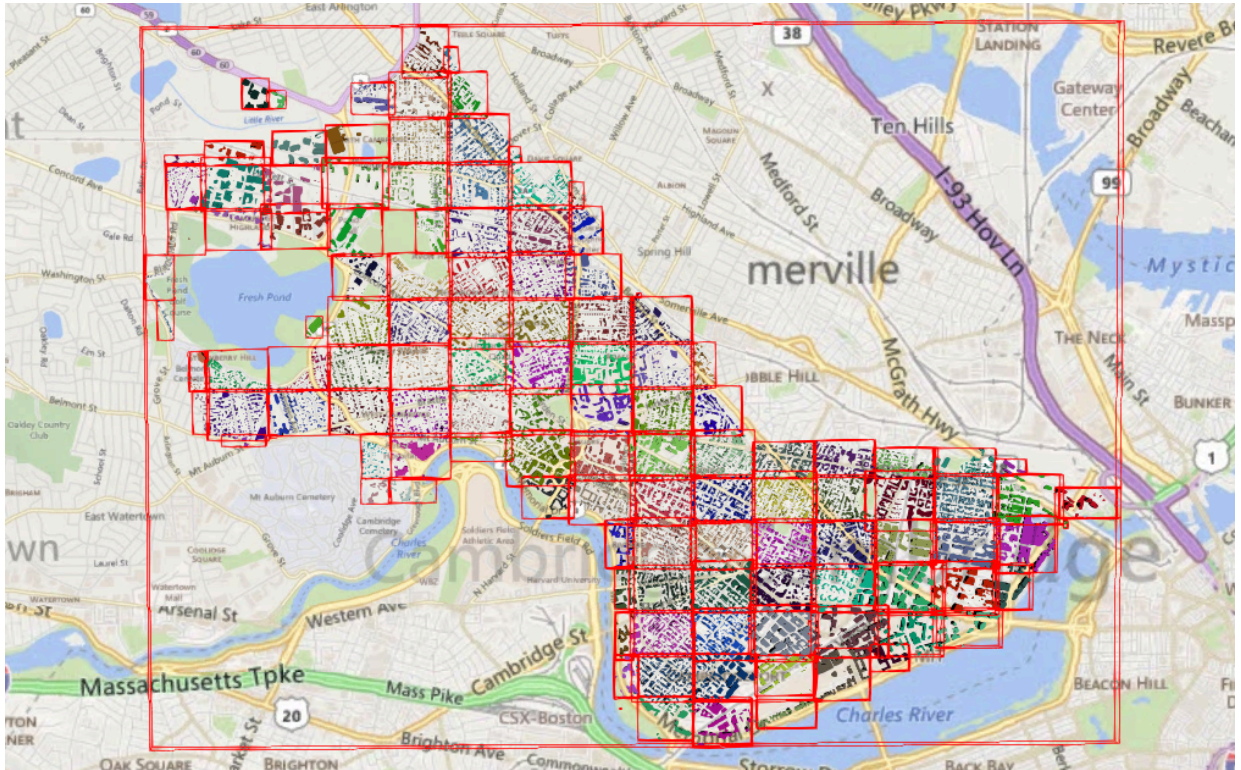


Figure A.23 — Building data from [CyberCity3D](#). Imagery data from [Bing Maps](#).

3D Tiles takes advantage of empty tiles: those tiles that have a bounding volume, but no content. Since a tile's externalAsset property does not need to be defined, empty non-leaf tiles can be used to accelerate non-uniform grids with hierarchical culling. This essentially creates a quadtree or octree without hierarchical levels of detail (HLOD).

A.12. Schema

- [content.schema.json](#)
- [glTF.3DTILES_tileset.schema.json](#)
- [node.3DTILES_tileset.schema.json](#)



ANNEX B (INFORMATIVE) CONTRIBUTORS



ANNEX B (INFORMATIVE) CONTRIBUTORS

Editors:

- Patrick Cozzi, [@pjcozzi](#), patrick@cesium.com
- Sean Lilley, [@lilleyse](#), sean@cesium.com
- Amanda Morgan, [@akmorgan99](#), amanda.morgan@cesium.com
- Adam Morris, [@weegeekps](#), adam@cesium.com

Acknowledgements:

- Matt Amato, [@matt_amato](#)
- Erik Andersson, [@e-andersson](#)
- Dan Bagnell, [@bagnell](#)
- Tamrat Belayneh
- Ray Bentley
- Björn Blissing
- Jannes Bolling, [@jbo023](#)
- Johan Borg
- Dylan Brown, [@Dylan-Brown](#)
- Sarah Chow, cesium.com/team/SarahChow
- Paul Connelly
- Volker Coors
- Erik Dahlström, [@erikdahlstrom](#)
- Henrik Edstrom
- Tom Fili, [@tfili](#)
- Leesa Fini, [@LeesaFini](#)
- Peter Gagliardi, [@ptrgags](#)

- Gabby Getz, [@gabbygetz](#)
- Ralf Gutbell
- Frederic Houbie
- Jeshurun Hembd, [@jjhembd](#)
- Xuan Huang, [@xuanhuang1](#)
- Marco Hutter, [@javagl](#)
- Daniel Krupka, [@krupkad](#)
- Ian Lilley, [@IanLilleyT](#)
- Janine Liu, [@j9liu](#)
- Don McCurdy, [@donmccurdy](#)
- Christopher Mitchell, Ph.D., [@KermMartian](#)
- Shehzan Mohammed, [@shehzan10](#)
- Claus Nagel
- Jean-Philippe Pons
- Andreas Plesch, [@andreasplesch](#)
- Carl Reed
- Kevin Ring, [www.kotachrome.com/kevin](#)
- Mark Schlosser, [@markschlosseratbentley](#)
- Scott Simmons
- Sam Suhag, [@sanjeetsuhag](#)
- Rob Taglang, [@lasalvavida](#)
- Stan Tillman
- Piero Toffanin, [@pierotofy](#)
- Bao Tran, [@baothientran](#)
- Samuel Vargas, [@samulus](#)
- Pano Voudouris
- Dave Wesloh
- Daniel Zhong, [@danielzhong](#)



ANNEX C (INFORMATIVE) REVISION HISTORY



ANNEX C (INFORMATIVE) REVISION HISTORY

DATE	RELEASE	EDITOR	PRIMARY CLAUSES MODIFIED	DESCRIPTION
2026-06-03	2.0.0	Adam Morris	all	This is the initial normative specification for 3D Tiles 2.0.